SCHOOL OF ENGINEERING MATHEMATICS AND TECHNOLOGY

Hey LoRA, it's Me: A Systematic Assessment of the Performance of Fine-Tuned Protein Language Models and Domain-Specific Engineered Features for the Downstream Task of Linear B-Cell Epitope Prediction

Harry Baylis

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree of Master of Science in Data Science in the Faculty of Science and Engineering.

Friday 4th September, 2026

Supervisor: Dr. Felipe Campelo

Declaration

This dissertation is submitted to the University of Bristol in accordance with the requirements of the degree of MSc in the Faculty of Engineering. It has not been submitted for any other degree or diploma of any examining body. Except where specifically acknowledged, it is all the work of the Author.

Harry Baylis, Friday 4th September, 2026

Contents

List of Figures

List of Tables

List of Additional Code

Abstract

B-cell epitopes are antigenic regions recognised by B-cell receptors that are important targets in vaccine and therapeutic development [1, 2, 3, 4]. A class of B-cell epitopes, linear B-cell epitopes (LBCEs) are sequences of amino acids adjacent in an antigen's primary structure that can be computationally predicted as a low-cost alternative for LBCE screening [3, 5, 6, 7].

Protein language models (pLM) provide abstract sequence representations that capture protein structure, functionality and evolutionary context that can be adapted to downstream tasks using fine-tuning [8, 9, 10, 11]. Parameter-efficient fine-tuning (PEFT) methods like low-rank adaptation (LoRA), enable task-specific adaptation at lower computational cost than full-fine tuning [12, 8].

Previous studies suggest fine-tuning pLMs can improve downstream predictive performance [13], including LBCE predictions for data-scarce pathogens using phylogeny-informed data [7]. This project hypothesised that pLM fine-tuning would improve the performance of per-residue classification predictions with phylogeny-aware datasets, compared with frozen pLMs and domain-specific features.

This project included the development, training, evaluation and final performance assessment of pLM pipelines for the downstream task of LBCE per-residue classification prediction using phylogeny-aware datasets. Predictions employing engineered Pfeature representations were also assessed. Full fine-tuning produced pathogen specific effects that were directionally inconsistent with fine-tuning mode and model size. Additionally, LoRA PEFT did not improve performance compared with frozen pLM embeddings. Overall the results do not support a consistent benefit from pLM fine-tuning for phylogeny-aware per-residue LBCE classification prediction across the pathogens tested.

Supporting Technologies

- Phylogeny-aware pathogen datasets were provided by Felipe Campelo based on the R package 'epitopetransfer' [14].
- Open-source Python libraries/packages including PyTorch, Hugging Face (Transformers, Datasets and PEFT), NumPy, SciPy, Pandas, scikit-learn, Biopython, fair-esm, Matplotlib, s3fs and boto3.
- The Pfeature software package was used to generate engineering protein features. Software downloaded from [15].
- Amazon Web Services (AWS) was used to process Python-based deep learning workflows on high-performance GPU instances and to store data remotely.
- Isambard-AI National AI Research Resource (AIRR) to process Python-based deep learning workflows on high-performance GPU instances and to store data remotely [16].
- GitHub and Git for version control and file storage.
- LaTeX to format this thesis, via the online service *Overleaf*.

Acronyms

AAC	:	Amino-acid composition
AIRR	:	National AI Research Resource
ANOVA	:	Analysis of variance
ATC	:	Atomic composition
AUC	:	Area under the curve
B	:	Billion
BTC	:	Bond composition
CLS	:	Classification token
CSV	:	Comma-separated values
CUDA	:	Compute Unified Device Architecture
DSIT	:	Department for Science, Innovation and Technology
EOS	:	End of sequence
FDR	:	False discovery rate
FN	:	False negative
FP	:	False positive
HAC	:	Hierarchical agglomerative clustering
HS	:	Holm-Sidak (multiple comparisons test)
LBCE	:	Linear B-cell epitope
LLMs	:	Large language models
LoRA	:	Low-rank adaptation
M	:	Million
MLM	:	Masked Language Modelling
MCC	:	Matthews correlation coefficient
MHCs	:	Major histocompatibility complexes
NA	:	Not applicable
PCA	:	Principal component analysis
PCP	:	Physicochemical properties
PEFT	:	Parameter-efficient fine-tuning
pLM	:	Protein language model
ProtT5-XL	:	Protein Transformer T5-Extra Large
ROC	:	Receiver operating characteristic
ROC-AUC	:	Receiver operating characteristic - Area under the curve
SE	:	Shannon-entropy
TN	:	True negative
TP	:	True positive
t-SNE	:	t-Distributed stochastic neighbour embedding

Acknowledgements

Thank you to Felipe Campelo for his guidance and support on this project.

The author acknowledges the use of resources provided by the Isambard-AI National AI Research Resource (AIRR). Isambard-AI is operated by the University of Bristol and is funded by the UK Government's Department for Science, Innovation and Technology (DSIT) via UK Research and Innovation; and the Science and Technology Facilities Council [ST/AIRR/I-A-I/1023].

Ethics statement: This project fits within the scope of ethics pre-approval process, as reviewed by my supervisor Felipe Campelo and approved by the faculty ethics committee as application 15208.

Chapter 1

Introduction

B-cell epitopes are specific amino acid configurations on an antigen's surface that are recognised and bound by B-cell receptors [1, 2, 3]. As epitope/B-cell receptor interactions play a part in the immune response [17], identifying epitopes is key to developing vaccines, therapeutic antibodies and immunodiagnostics [3, 4]. B-cell epitopes are broadly categorised into linear and conformational, where linear B-cell epitopes (LBCEs) are short sequences of amino acids that are adjacent in an antigen's primary sequence, while conformational B-cell epitopes are formed from non-adjacent amino acids brought together by the folding (tertiary structure) of an antigenic protein [5, 6].

Epitopes can be identified through experimental techniques such as X-ray crystallography, nuclear magnetic resonance spectroscopy and peptide microarrays [4, 18]. These methods underpin much of the scientific understanding of protein biochemistry, in turn generating vast amounts of data archived within various open-source protein databases [13, 11]. Leveraging this data, computational methods to predict epitope sequences have been developed, often circumventing the cost and resource implications of traditional experimental techniques [3, 5, 6, 7].

One example is the use of protein feature engineering tools such as and Pfeature [18] that integrate evolutionary data and experimentally-informed residue annotations to produce numeric descriptors or 'features' of amino acids in sequence. These features can in turn be used as inputs to machine learning models to predict epitope sequences [18].

More recently, deep learning approaches from large language models (LLMs) have been used to develop protein language models (pLMs) that learn the vocabulary and sequence patterns of extensive corpora of protein sequences [6, 13, 8]. Using these sequences as inputs, pLMs generate residue-level embeddings that represent a rich semantic meaning of a protein with long-range context [19, 20]. These embeddings capture both the linear sequence of a protein but also a representation of its general structural, functional and evolutionary context [8, 9, 10].

pLMs can be adapted for epitope prediction through supervised transfer learning [3, 7]. A simple example is replacing the pLM's classification head with one trained to classify epitopes [7]. To improve predictive performance, the pLM's weights can be updated using a new target objective so that the model is better adapted to a specific task [13, 7]. This process is known as fine-tuning. As pLMs can contain billions of parameters, updating every parameter can be inefficient due to the required computation and training time [13, 11]. Parameter-efficient fine-tuning (PEFT) aims to lower this overhead by instead of updating all parameters, the weights of smaller matrices, or adapters, are fine-tuned and injected into the model architecture to impart a change to the original pre-trained weights, resulting in fine-tuned model at a lower training cost [13, 12]. Previous work has shown that low-rank adaptation (LoRA), a PEFT method, provides competitive prediction performance to full fine-tuning for protein prediction tasks whilst training only 0.25% of the model's parameters [13].

As epitope sequences are often highly conserved between species due to genetic similarity from common ancestry, particular care must be taken when segregating epitope sequences for model development and testing to prevent data leakage and artificial predictions [17, 4, 7]. Previous work successfully fine-tuned a pLM using used phylogeny-informed data from data-rich pathogens to generate accurate predictions for data-scarce pathogens [7]. Although PEFT has been applied to the transfer learning of pLMs to epitope [21] and antigen specificity predictions [11], evidence of its application to phylogeny-informed LBCE prediction is unknown.

Aims:

- Using the pLMs referenced by Schmirler et al., [13], complete a comparative assessment of pLMs deployed for transfer learning of LBCE prediction, incorporating phylogeny-aware datasets during training and testing.
- Compare to input representations generated from domain-specific feature engineering tool (Pfeature), incorporating phylogeny-aware datasets during training and testing.

Objectives:

- Develop Python pipelines to compare transfer learning approaches for the purpose of **residue-level LBCE prediction classification** using a series of pathogen datasets for training and testing:
  - Full-fine-tuning:
    - Re-train the pLM using data from a higher taxonomic level than the target pathogen, excluding data from the target pathogen (termed higher-level data).
    - Train a neural network classifier for LBCE prediction using data from the same taxon as the target pathogen, excluding the target pathogen (lower-level data).
    - Generate residue-level embeddings only for the target pathogen using the fine-tuned pLM (no classification head).
    - Pass these embeddings through the pre-trained classifier to generate LBCE classification predictions.
  - PEFT:
    - Employ the same process as full-fine-tuning above with the exception that PEFT is used to fine-tune the pLM, specifically using LoRA as the PEFT method.
  - Base (as-standard) embedder:
    - Using lower-level data, generate residue-level embeddings from the pre-trained pLM with no fine-tuning (pre-trained weights) with no classification head.
    - Train a neural network classifier for LBCE prediction using these embeddings.
    - Generate embeddings for the target pathogen by using the base/pre-trained pLM. Pass these embeddings through the pre-trained classifier to generate LBCE predictions (positive or negative).
- Develop a Python pipeline using a domain-specific feature engineering tool (Pfeature).
  - Generate numeric representations of lower-level data using the feature engineering tool.
  - Train a neural network classifier for LBCE prediction using these representations.
  - Create representations for the target pathogen and predict the LBCE classification using the pre-trained classifier.
- Analyse the performance of each pipeline with respect to performance of fine-tuning method and model size (pLM methods only) and between pLM vs domain-specific representation methods.

Achievements:

- **ESM-2 pLM**: full fine-tuned, PEFT (LoRA) and base embedder pipelines completed and predictions generated for 12 pathogens using 8M, 35M and 150M parameter models on AWS cloud-based GPU instances.
- **ESM-2 pLM**: full fine-tuned, PEFT (LoRA) and base embedder pipelines completed and predictions generated for 12 pathogens using the 650M parameter on Isambard AI Phase 2 GPU instances.
- **ProtT5-XL**: full fine-tuned, PEFT (LoRA) and base embedder pipelines completed and predictions generated for 1 pathogen using the 1.5B parameter model on Isambard AI Phase 2 GPU instances.
- **Pfeature**: engineered-feature pipeline completed and predictions generated for 12 pathogens.
- A comparative assessment across model size, model type and representation generation methods completed.

Chapter 2

Background

The role of epitopes in the immune system

B lymphocytes (B-cells) play a key role in the adaptive immune system, contributing to the immune response against pathogens [1, 2]. B-cells are activated when cell surface proteins called B-cell receptors bind complementary regions on pathogen-derived antigens. In the immunological context, pathogens are often referred to as antigens [6]. The antigen regions complementary to B-cell receptors are termed 'epitopes' and are broadly grouped into linear and conformational epitopes [5, 6]. Linear epitopes are continuous polypeptide sequences, and conformational epitopes are discontinuous amino-acids of one or more residues in sequence folded together through the antigen's tertiary structure, forming a specific B-cell receptor binding domain [5, 6, 22] (Figure 2.1, taken from [22]). Linear B-cell epitopes (LBCEs) are estimated to make up 10% of B-cell epitopes [5], however due to their presence in a protein's primary structure, are conventionally better predisposed for computational methods by harnessing experimentally-characterised open source primary sequences [4].

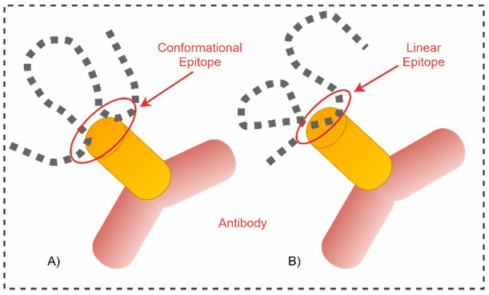


Figure 2.1: Schematic of conformational A) and linear B) B-cell epitopes.

Other epitope prediction targets include T-cell epitopes which are bound by T-lymphoctyes (T-cells) receptors [21]. Unlike B-cell epitopes, T-cell epitopes are proteolytically-cleaved foreign peptide fragments that are presented to T-cells on major-histocompatibility complexes (MHCs) of antigen presenting cells [21, 1]. Ultimately, B-cells detect epitopes on antigens in their native form whereas T-cells detect epitopes as immunologically processed peptide fragments. Despite both detecting specific epitope sequences, B-cells and T-cells play distinct yet harmonized roles in the adaptive immune system [23, 1]. This project focuses on the prediction of linear B-cell epitopes (LBCEs).

Computational methods for epitope prediction

Early computational methods for LBCE prediction compared the similarity of known and unknown sequences, known as multiple sequence alignment (MSA) [18, 13]. MSA assumes a likelihood of sequence conservation through evolution, informing an increased chance of functional (immunogenic) similarity [18, 13]. Consequently, MSA is limited to the comparison of functionally annotated sequences and can struggle with novel sequences such as those from understudied or emerging pathogens [18].

As an LBCE must be accessible to a B-cell receptor, protein physiochemical properties can infer epitope likelihood through properties such as hydrophilicity, charge and surface accessibility [4, 6]. As such, protein feature engineering tools such as Pfeature containing extensive lookups of different physiochemical properties for given sequence inputs, along with some contextual information on neighbouring residues [18]. Critically, these tools numerically encode this information so that it can be used as input to computational models [18]. The Pfeature software package includes over 200,000 features, divided into 'composition, binary profiles, evolutionary information, structural features, patterns, and model building' modules. The input to Pfeature is a FASTA file, a plain text file that includes a specific FASTA file string header followed by amino acid letter codes [18, 15]. A limitation of protein feature engineering tools is a lack long range context, often limited to a few neighbouring residues. Given the array of available features, this could raise questions of which features to select for a certain task, as well as how to weight the prominence of certain features in relation to one another.

Protein language models

In contrast, protein language models (pLMs) have been shown to deliver complex protein representations [6, 13, 8]. Similar to how the choice and order of words in a sentence coveys meaning, the sequential order of amino acids impacts protein structure and resulting function [10]. Through learning sequence patterns and long-range relationships between residues in sequence, protein language models (pLMs) have enabled detailed representation of structure, function and evolutionary context [8, 9, 10]. As with other deep learning models, the expressiveness of a pLM has been shown to trend with pre-training data volume and model parameter size, at the expense of increased computational overhead [8, 9, 13]. Similar to large-language models (LLMs), pLMs contain encoder architectures with multi-head self-attention which allowing each residue to 'learn' the significance every amino-acids in sequence and its contextual relationship [3, 10].

pLMs are pre-trained on large corpora of protein primary sequences using unsupervised masked language modelling (MLM) [8]. Here a pLM-specific mapping of each amino acid's one-letter code to a corresponding numerical input identifier, known as tokens, are systemically masked, as the model learns to predict the masked token in context of the input sequence [13, 8]. Model weights are adjusted using backpropagation to minimise a cost function associated with the probability of the predicted amino acid and the true masked amino acid [8, 9]. During inference, a forward pass is made through the encoder network: multiple matrix multiplications and non-linear transformations are applied to each amino acid token according to the model's pre-trained weights and architecture. This process creates an embedding matrix of size L * d from its final hidden layer, where L is the length of the amino acid input sequence and d is the size of the embedding. These embeddings provide a contextualised representation of every amino acid in latent space based on the pLM's pre-training data [9].

One example is the evolutionary-scale modelling 'ESM-2' pLM [24, 8]. ESM-2 models are pre-trained on UniRef50 protein data with model parameter size checkpoints between 8 million (M) and 15 billion (B). Each checkpoint has a different number of transformer layers ranging between 6 and 48 and hidden layer embedding dimensions between 320 and 5,120 as detailed in Table 2.1, taken from [25]. A supplementary model, ESMFold, that produces 3D structure predictions from protein sequences, leverages the attention patterns of ESM-2 to create residue contact maps that correspond to a protein's tertiary structure that with additional transformations can produce an atomic level 3D structure [8]. ESM-2 has been fine-tuned to produce LBCE predictions [7], residue-level linear and conformational B-cell epitope predictions [26], adeno-associated virus vector predictions [19] and defensive protein classifiers [20].

Table 2.1: ESM-2 checkpoints

Checkpoint	Transformers (n)	Parameters (n)	Dataset	Embedding Dim.
`esm2_t48_15B_UR50D`	48	15B	UR50/D 2021_04	5120
`esm2_t36_3B_UR50D`	36	3B	UR50/D 2021_04	2560
`esm2_t33_650M_UR50D`	33	650M	UR50/D 2021_04	1280
`esm2_t30_150M_UR50D`	30	150M	UR50/D 2021_04	640
`esm2_t12_35M_UR50D`	12	35M	UR50/D 2021_04	480
`esm2_t6_8M_UR50D`	6	8M	UR50/D 2021_04	320

Another is the ProtTrans suite of models, pre-trained on data from UniRef50 and BFD [27, 28]. The ProtT5-XL model, contains 24 transformer layers, 3B parameters and an encoder-decoder that can generate sequence predictions. One difference to ESM-2 is that ProtT5 was trained using a denoising

objective that corrupts spans of tokens in sequence as opposed to masking individual tokens, typical of MLM [27, 28]. Like ESMFold, an protein anatomic-level structure prediction is available, ProstT5 [27, 28]. An example of its use is in the prediction of LBCEs by training a Support Vector Machine classifier on experimentally validated epitopes dataset embedded by ProtT5 [29].

Task-specific transfer learning

Despite the large pre-training datasets and generality of pLMs, the pre-trained weights are unlikely to be adequately aligned to a given prediction task. Model fine-tuning involves making small updates to the model weights in accordance with a new training objective with data relevant to the target task [11]. An advantage is that target domain data may be scarce and fine-tuning serves to complement an already well characterised and generalised representation of protein 'language' without the significant upfront training costs of pre-training [11, 19]. Despite this, full fine-tuning can be computationally demanding, especially where pLM sizes reach several billion parameters [12, 8]. As increasing model size has been shown to consistently yield improved predictive performance, efficiently adapting models for downstream tasks against the trade-off in training cost is a field of research interest [30]. Consequently research efforts have been made into a branch of fine-tuning methods known as parameter-efficient fine-tuning (PEFT). PEFT comprises various approaches in terms of how trainable parameters are structured and positioned in a pLM's architecture, however the commonality is that they apply a 'delta' to a set of the pLM's parameters [30]. These 'deltas' are known as 'adapters', that are either intrinsic in the pLM architecture or are new matrices that are added into the pLM [30].

Low-rank adaptation

One such method is low-rank adaptation (LoRA), which hypothesises that the change in weights (delta) during tuning has a 'low-intrinsic rank', that is, this change has the ability to be reduced into lower dimensions without losing any fundamental information. Specifically, LoRA adds two additional matrices (represented as A and B below) to the self-attention modules within each transformer block [12, 30].

This is summarised in [12]: considering a pre-trained weight matrix W_θ represented by $W_\theta \in \mathbb{R}^{d\times k}$ of dimensions r by k, its update can be represented by a low-rank decomposition $W_\theta + \Delta W = W_\theta + BA$ where B and A are represented as $B \in \mathbb{R}^{d\times r}$ and A as $A \in \mathbb{R}^{r\times k}$ where the parameter rank $rmin(d, k)$. The parameter rank r can be set to the full rank of the original weight weight matrices to provide the full expressiveness of the original model as if were being fully fine-tuned, whereas reducing r decreases the expressiveness with the benefit of reducing the dimension of the adapter matrices and the training cost.

When training using LoRA, the pre-trained weight matrix W_θ is frozen whereas A and B contain tunable parameters as detailed in the schematic in 2.2. The main cost benefits of LoRA are memory and storage usage, however LoRA has been shown to accelerate training time with larger parameter models by reducing the need to compute, store, and update gradients for the majority of the model's parameters [12, 13, 30].

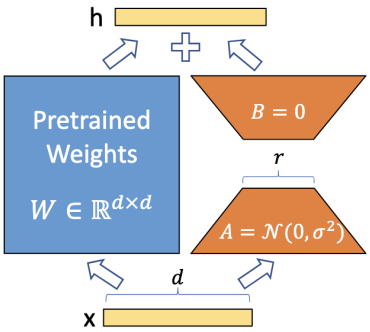


Figure 2.2: Reparametrisation using LoRA. x is the input and h is a function of x after a forward pass through the the model. A representation of the pre-trained weights is shown on the left and a representation of the addition of adapter matrices A and B in fine-tuning on the right. r represents the rank of the weight decomposition.

Performance gains with pLM fine-tuning

An empirical analysis of fine-tuning methods for three protein language models ESM-2, ProtT5 and Ankh for different tasks was completed by Schmirler et al. [13]. The authors showed that task-specific fine-tuning of pLMs increased performance in 59 of the 64 combinations of pLM model checkpoint and prediction tasks, versus using the frozen embeddings from the pre-trained 'base' pLMs. From these results, they recommended supervised fine-tuning whenever using pLM embeddings were being used for a downstream prediction task. Figure 2.3, taken from [13], presents the paper's results comparing the percentage difference of the mean performance gains/losses of fine-tuning different pLMs for eight downstream tasks against using the base pLM for embedding generation. They evaluated different performance measures depending on the downstream task, as defined by the original dataset developers. This included Spearman rank correlation for protein-level regression tasks, n-class accuracy for protein level classification tasks and n-class accuracy for per-residue classification and regression tasks. In terms of training time, LoRA provided an approximate 4.5 fold decrease in training time versus full fine-tuning for the ESM-2 3B parameter model [13].

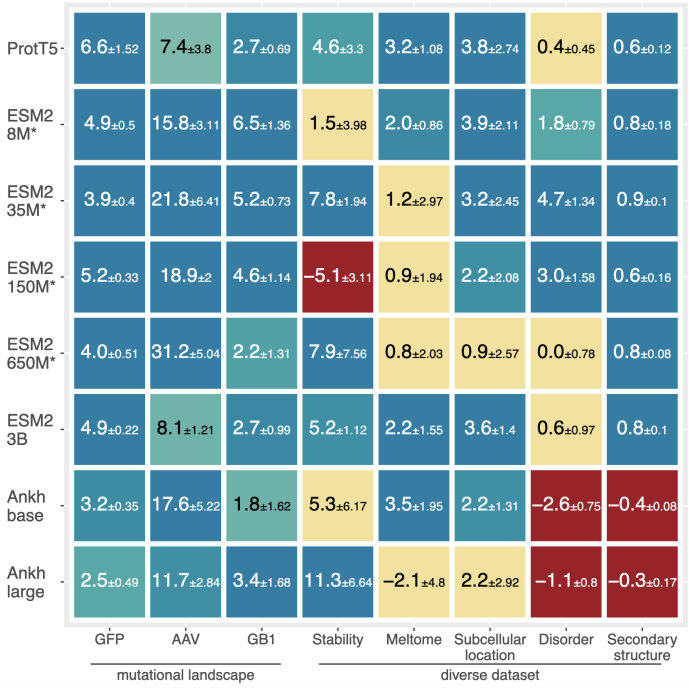


Figure 2.3: pLM % difference in performance gains/losses for fine-tuned pLMs versus the base pLM for a range of downstream prediction tasks. Models with an asterisk were fully fine tuned, whereas those without were fine-tuned using LoRA PEFT only.

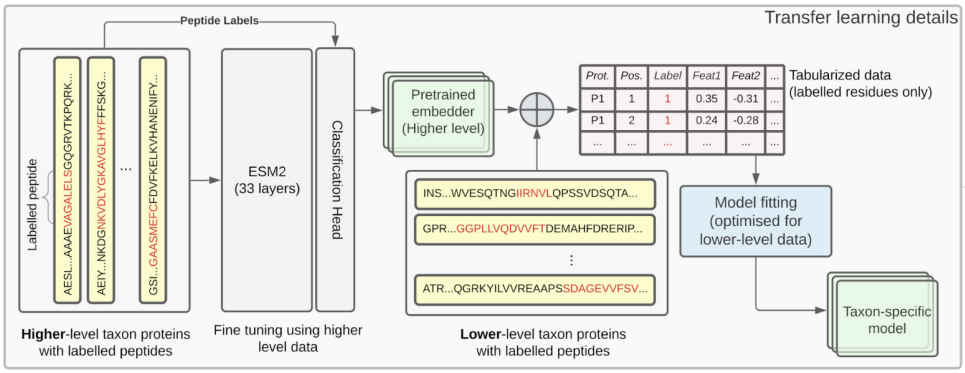


Figure 2.4: Pipeline schematic showing ESM-2 fine-tuning for downstream LBCE prediction using phylogeny-informed data.

Additional performance gains with phylogeny-aware datasets

Furthermore, Leite et al. [7] successfully leveraged phylogenic-informed datasets to develop a protein-level pipeline containing a fine-tuned 650M parameter ESM-2 model and random-forest classifier epitope sequences from data-scarce pathogens. Their work assimilated previous findings that carefully consider preventing data leakage through genetic similarity from common ancestry to generate LBCE predictions for data-scarce pathogens using training data from higher but genetically-related taxon levels [17, 4, 7]. A portion of a schematic taken from [7] details the transfer learning process flow (Figure 2.4). Results showed that the phylogeny-informed pipeline provided AUC performance gains (between +0.09 and +0.123) over both generalist LBCE predictors and a recent taxon-specific predictor. This project therefore aims to complete a systematic performance assessment of phylogeny-informed LBCE prediction using pLM task-specific fine-tuning, with comparison to protein representations using domain-specific features. This project hypothesises that pLM fine-tuning will improve the performance of per-residue classification predictions with phylogeny-aware datasets, compared with frozen pLMs and domain-specific features.

Chapter 3

Methods

3.1 Data and version control

3.1.1 Data

Protein/epitope data was supplied by Felipe Campelo, generated using R package 'Epitopetransfer' [14]. For each pathogen there were 3 csv files for higher-level data, lower-level data and target data. As detailed in Chapter 1, the higher-level data contained data from a higher taxonomic level than the target pathogen, excluding data from the target pathogen (termed higher-level data). Lower-level data contained data from the same taxon as the target pathogen, excluding the target pathogen. Target data contained data for the specific pathogen for final model inference/performance assessment. Each csv file was a vertically stacked table of amino acids in sequence order for different proteins, with an epitope 'Class' label from experimentally derived data. Each residue was labelled either positive '1', negative '-1' or unknown/untested 'NA'. To support cross-validation for pLM fine-tuning and classifier training using the higher-level and lower-level data respectively, each protein had been pre-assigned to one of five training/validation 'info_splits' folds to prevent data leakage. A breakdown of the column names for each table is shown in table 3.1. A breakdown of the different pathogens and the number and proportion of negatively and positively labelled samples for each pathogen is shown in table 3.2. For generating engineered features, an 'info_window' column containing up to 7 amino acids either side of the given residue was used.

Table 3.1: Phylogenic data column structure

	Info_protein_id	**Info_pos**	**Info_AA**	**Info_PepID**	**Info_group**	**Info_split**	**Info_window**	**Class**
Higher-level	X	X	X	X	X	X	X	X
Lower-level	X	X	X	X	X	X	X	X
Target	X	X	X	X			X	X

3.1.2 Version control

Python scripts, outputs and environment files are saved in the following GitHub repository: `https://github.com/harryb257/MScProject`. GitHub was used for version control throughout the project.

3.2 Experimental design

Pipelines were developed for 3 fine-tuning modes: pre-trained pLM with no fine-tuning (base), LoRA fine-tuning and full fine-tuning. 12 pathogens were tested using all 3 modes for ESM-2 parameter sizes of 8M, 35M, 150M and 650M and for 1 pathogen using all 3 modes with ProtT5-XL. 12 pathogens were tested using a pipeline with Pfeature representations. Table 3.3 summarises the pLM checkpoint and Pfeature test runs completed for each pathogen.

Table 3.2: Dataset composition across pathogen-specific taxonomic levels used in this project. The negative-to-positive ratio is calculated as nNeg/nPos.

Pathogen	Level	Scientific Name	Rank	nNeg	nPos	nTot	nProts	Neg:Pos
Bunyaviricetes	Higher	Orthornavirae	kingdom	8728	4516	13244	4175	1.93
	Lower	Polyploviricotina	subphylum	108	335	443	267	0.32
	Target	Bunyaviricetes	class	92	103	195	69	0.89
Campylobacter jejuni	Higher	Pseudomonadati	kingdom	682	2302	2984	1561	0.30
	Lower	Campylobacterales	order	54	66	120	48	0.82
	Target	Campylobacter jejuni	species	57	29	86	23	1.97
Clostridioides difficile	Higher	Bacillota	phylum	479	368	847	321	1.30
	Lower	Clostridia	class	95	93	188	31	1.02
	Target	Clostridioides difficile	species	43	33	76	8	1.30
Dengue virus	Higher	Amarillovirales	order	633	635	1268	325	1.00
	Lower	Euflavivirus	subgenus	429	567	996	134	0.76
	Target	Dengue virus	no rank	821	1422	2243	534	0.58
Legionellales	Higher	Pseudomonadati	kingdom	623	1971	2594	1359	0.32
	Lower	Gammaproteobacteria	class	154	413	567	250	0.37
	Target	Legionellales	order	12	13	25	21	0.92
Mycobacterium leprae	Higher	Bacillati	kingdom	755	600	1355	444	1.26
	Lower	Mycobacterium	genus	415	413	828	339	1.00
	Target	Mycobacterium leprae	species	23	40	63	21	0.58
Mycoplasmoidaceae	Higher	Bacteria	domain	793	2400	3193	1788	0.33
	Lower	Bacillati	kingdom	1186	1027	2213	802	1.15
	Target	Mycoplasmoidaceae	family	12	37	49	7	0.32
Neisseria gonorrhoeae	Higher	Pseudomonadati	kingdom	723	2200	2923	1510	0.33
	Lower	Betaproteobacteria	class	50	175	225	101	0.29
	Target	Neisseria gonorrhoeae	species	20	15	35	20	1.33
Orthoparamyxovirinae	Higher	Orthornavirae	kingdom	8707	4639	13346	4319	1.88
	Lower	Mononegavirales	order	184	234	418	150	0.79
	Target	Orthoparamyxovirinae	subfamily	37	81	118	42	0.46
Orthopoxvirus	Higher	Viruses	acellular root	10078	6753	16831	5719	1.49
	Lower	Bamfordvirae	kingdom	52	188	240	109	0.28
	Target	Orthopoxvirus	genus	14	21	35	16	0.67
Pasteurellaceae	Higher	Pseudomonadati	kingdom	623	1971	2594	1359	0.32
	Lower	Gammaproteobacteria	class	150	375	525	240	0.40
	Target	Pasteurellaceae	family	17	49	66	31	0.35
Yersinia pestis	Higher	Pseudomonadati	kingdom	710	2163	2873	1496	0.33
	Lower	Enterobacterales	order	58	210	268	127	0.28
	Target	Yersinia pestis	species	13	19	32	5	0.68

3.3 pLM pipeline development

3.3.1 Overview

The project utilised two pLMs referenced within [13], ESM-2 and ProtT5. The Python scripts developed for the pLMs were identical where possible, with the exception of the deviations detailed later in the section. Depending on the deployment infrastructure (AWS or Isambard-AI Phase 2) updates were also made to accommodate file storage/transfer.

Each pipeline was contained three Python scripts: `workflow.py`, `train_pipeline.py`, and `utils.py`. `workflow.py` served as the orchestration layer, iterating over the relevant pathogen, training mode (pLMs only), and checkpoint (pLMs only, and where applicable). For each configuration, `workflow.py` called the `esm2_pipeline` function implemented in `train_pipeline.py`. This function contained the logic required to perform a single training and inference iteration, including calls to supporting functions defined in `utils.py`. In summary, `workflow.py` controlled the experimental configurations, `train_pipeline.py` executed the corresponding pipeline, and `utils.py` provided supporting functionality. An overview of `train_pipeline.py` is presented in Figure 3.1.

3.3.2 Implementation and data storage

Python scripts for the 8M, 35M and 150M parameter ESM-2 models were deployed on AWS using the EC2 instance configuration detailed in 3.4. The Python virtual environment used containing the package requirements/dependencies is saved in the GitHub repository. For file transfer and storage, local EC2 storage was for temporary storage prior to an S3 bucket for long-term durable storage. The instances

Table 3.3: Models and engineered features evaluated for each pathogen.

Pathogen	pLM (model / number of parameters)					Engineered Features
	ESM-2				ProtT5	Pfeature
	8M	35M	150M	650M	1.2B	
Bunyaviricetes	X	X	X	X		X
Campylobacter jejuni	X	X	X	X		X
Clostridioides difficile	X	X	X	X		X
Dengue virus	X	X	X	X		X
Legionellales	X	X	X	X		X
Mycobacterium leprae	X	X	X	X		X
Mycoplasmoidaceae	X	X	X	X		X
Neisseria gonorrhoeae	X	X	X	X		X
Orthoparamyxovirinae	X	X	X	X		X
Orthopoxvirus	X	X	X	X	X	X
Pasteurellaceae	X	X	X	X		X
Yersinia pestis	X	X	X	X		X

accessed pathogen data via an S3 buckets with separate directories for each pathogen (as per the supplied dataset).

Python scripts for the ESM-2 650M model and ProtT5-XL half precision version (1.5B parameters) were deployed on Isambard-AI Phase 2 using 1 NVIDIA GH200 Grace Hopper Superchip. An sbatch shell script was used to connect to a compute node and control the run. A Conda virtual environment with required dependencies is saved in the GitHub repository. The environment included a PyTorch version (2.13) compatible with the CUDA version of the installed NVIDIA driver.

Table 3.4: AWS instance configuration

Configuration	**Value**
Instance type	g5.2xlarge
Amazon Machine Image (AMI)	amazon/Deep Learning OSS Nvidia Driver AMI GPU PyTorch 2.12 (Ubuntu 24.04) 20260725
Security group	Created on instance launch; inbound SSH (TCP port 22) and unrestricted outbound traffic
Network	VPC with subnet
Availability zone	eu-north-1b (Stockholm)
Elastic Block Store (EBS)	100 GiB
Identity and Access Management (IAM) role	S3 access

3.3.3 ESM-2 - AWS deployment

The following section details the processing steps contained within the Python scripts `train_pipeline.py` and `utils.py'`.

Pathogen and checkpoint identification, file handling, and random seed initialisation

The `esm2_pipeline` function in `train_pipeline.py` performs the initial configuration of each pipeline run. The pathogen name and checkpoint name are extracted from the S3 bucket path and used to construct run-specific output filenames. A local output directory is then created for storing intermediate and final files. The S3 destination for copying these files is also configured. An `S3FileSystem` object is initialised, and the `select_datasets` function is used to identify the datasets required for the run from the S3 bucket. Datasets are assigned to the corresponding pipeline-level variables based on their S3 paths.

The Hugging Face tokenizer and ESM2 model are loaded according to the selected checkpoint and training mode. For fine-tuning methods, the `load_esm_model_classification` function is called to ini-

tialise the ESM2 model with a trainable token-classification head. For LoRA-based fine-tuning, trainable LoRA adapters are additionally injected according to the configuration as detailed in [13]. The model is then transferred to the GPU.

Fine-tuning - preprocessing higher-level data

For fine-tuning, the higher-level data csv file is preprocessed using the `preprocess_csv` function ready for batching by the Hugging Face `Trainer`. This function converts the csv file into a Pandas DataFrame, removes rows without an Info_split label, renames 'Class' to 'label', remaps -1 to 0 for alignment to PyTorch naming conventions, aggregates sequences, labels and positions by protein_id, applies a sliding window using `sliding_window` and deletes unlabelled rows with `delete_unlabelled_rows`. Unlabelled positions are remapped as '-100' to ensure masking by the PyTorch entropy loss function within `Trainer`. `sliding_window` breaks protein sequences into max sequence length of 1024 residues using an overlap of 512 residues to the prior (left hand side) sequence. Each new window is inserted into the DataFrame as a new sub-split. Note that batch size for fine-tuning and classifier training was set to 16 for all configurations based on available GPU vRAM.

Fine-tuning - creating cross-validation folds and converting to datasets

A dictionary with key-value pairs of the `Info_split` fold and preprocessed DataFrame split is created. The number of training and cross validation folds is specified by a global `fine_tune_val_folds` parameter. This dictionary also contains a key value-pair of all proteins from the original preprocessed DataFrame for final trainer fine-tuning. Each fold is converted to a PyTorch dataset class with a call to the `preprocess_higher_level` function. This function maps each row to the checkpoint tokenizer and pads the labels with a -100 label at the start of the sequence, representing the added start CLS input id token and a -100 token representing the end of sequence (EOS) token, then pads using -100 for the remaining positions to the max 1026 sequence length. An attention mask, padded to the max sequence length, is automatically created by the tokenizer to ensure the trainer does not attend to any padded positions after the actual residue positions + EOS token. The number of trainable parameters are calculated and saved locally.

Fine-tuning - Hugging Face trainer

Fine-tuning leverages the Hugging Face `Trainer` class with custom features as per the `TrainingArguments` class and custom validation metrics using a `compute_metrics` function. fine-tuning evaluation uses 3 different `Info_split` folds. For example, when training/evaluating fold 1, all proteins not labelled `Info_splits: split_01_20` train the model and those labelled split_01_20 are held out to evaluate the model using the custom `compute_metrics` function. Training/evaluation is performed over 3 epochs per training/cross-validation fold. 5 evaluation folds were available for each dataset, however 3 folds were used due to the overall training time especially with larger models. After evaluation, a fresh fine-tuning process is is carried out using all pre-processed higher-level data. This final model along with the validation and final training metrics and logs are saved locally.

Classification head - Preprocessing lower-level data and embedding generation

A custom PyTorch `SequenceDataset` Dataset class converts a DataFrame into a PyTorch Dataset to access one protein and its data at a time. `batch_create` combines samples into batches and leverages the model's tokenizer and model to generate embeddings for each sequence. A custom `collate_fn` function ensures that proteins with different lengths relative to other proteins in the batch can be processed into tensors.

In more detail, `batch_create` converts tokenised sequences into embeddings using the loaded pLM (with pre-trained or fine-tuned weights). These embeddings are padded into to a max sequence length of 1024, and the start CLS and final padded position from the embeddings are removed. Labels are converted to tensors and padded to the length of the longest tensor in the batch. This implementation retains the EOS token position if the sequence is less than the max length. However if retained, its corresponding label is assigned a value of -100 during padding. This masking ensures that the EOS token does not contribute to the classification loss during training or evaluation. `batch_create` returns a dictionary of padded embeddings and associated padded labels. Finally `create_datasets_for_clf` produces training and validation dictionaries for each `Info_split` fold calling `SequenceDataset` and `batch_create` functions.

Classification head - Loss function weighting

PyTorch `CrossEntropyLoss` takes a parameter `weight` that a imparts a rescaling to each class. The relative weights of each class in the lower-level data is calculated using `class_weighting_for_clf`. This generates a 2 x 1 tensor of negative, positive residue weights, excluding any rows without an `info_split` labelling. Since the classifier was trained using a weighted cross-entropy loss based on the lower-level class weightings, the fold specific weightings were not calculated for each fold. Consequently, the weightings applied to each fold as a result of the dataset level could have resulted in the loss function imparting a different relative weightings to the classes across folds. However as the final classifier used for target inference was trained using all lower-level data, the cross entropy loss function weighting was therefore representative of the training data used to train the final model.

Classification head - Cross-validation and training

A PyTorch neural network `PerResidueClassifier` is instantiated: a 2-layer classifier with an input dimension as per the dimension of the `last_hidden_state` of the current ESM2 checkpoint. The first linear layer is followed by a ReLU activation function to the input with dropout (0.2) and passed to a second linear layer with output dimension of 2, representing the logits for the negative and positive classes. These operations are combined through the class' `forward` method, initiating a forward pass through the network.

The training loop uses `train_one_epoch`, `val_one_epoch` and `train_classifier` functions. `train_classifier` orchestrates the training and validation process, iterating over 20 training and validation epochs for each `Info_split` fold, generating a trained classifier and a per-residue DataFrame of labels, probabilities and predictions for unmasked residues. Within `train_one_epoch`, the classifier is trained by iterating over each batch. Per batch the network generates two class logits for every residue which are passed to the PyTorch loss function to calculate the training objective. Gradients are then calculated through backpropagation and classifier weights are updated accordingly and the loss and accuracy for the epoch is returned.

`val_one_epoch` assesses the performance of the dataset on the hold out data for each fold. In addition to loss and accuracy, it calculates additional metrics and generates per residue labels, probabilities and predictions. `train_classifier` returns the metrics for each fold and epoch along with per-residue labels, predictions and probabilities. The mean and standard deviation of the training and validation metrics across folds are subsequently calculated. The highest mean AUC value epoch is then used as the maximum epoch to train the final classifier using a complete lower-level dataset with the `train_final_classifier` function. The `model_state_dict()` (trained weights) and training metrics are saved locally.

Predictions using target data

The target dataset is preprocessed into a by-protein aggregated DataFrame, applying `sliding_window` and `delete_unlabelled_rows` functions. Embeddings and associated labels are created using `Sequence Dataset` and `batch_create`, employing the relevant base or fine-tuned ESM-2 model. These embeddings are then classified using the pre-trained `PerResidueClassifier`. Test metrics, labelled per residue labels, probabilities, predictions and confusion matrix for the per-residue classifications are saved locally. Local files are transferred to an S3 bucket and are removed from the local drive after successful S3 upload.

Residue-alignment verification

To verify the correct alignment of each sequence residue to its associated label after mapping the preprocessed csv file to the ESM-2 tokenizer, a retrospective alignment check was performed. This was achieved by inserting a temporary code block that compared label/residue alignment pre and post tokenisation and label preprocessing using the `preprocess_higher_level` function, which confirmed that labels were being correctly aligned to their associated input_id. The same check was also carried out for tokenisation, embedding generation and label preprocessing as part of the lower-level data `batch_create` function. This code blocks and resulting csv files for one protein are saved in the GitHub repository.

3.3.4 Pipeline deviation for ESM-2 fine-tuned models using AWS

To enhance the degree of information derived from the experiments, code updates were made to include additional outputs and performance metrics from classifier training and target prediction. As model fine-tuning had already been completed, the updates were incorporated into script versions with amendments

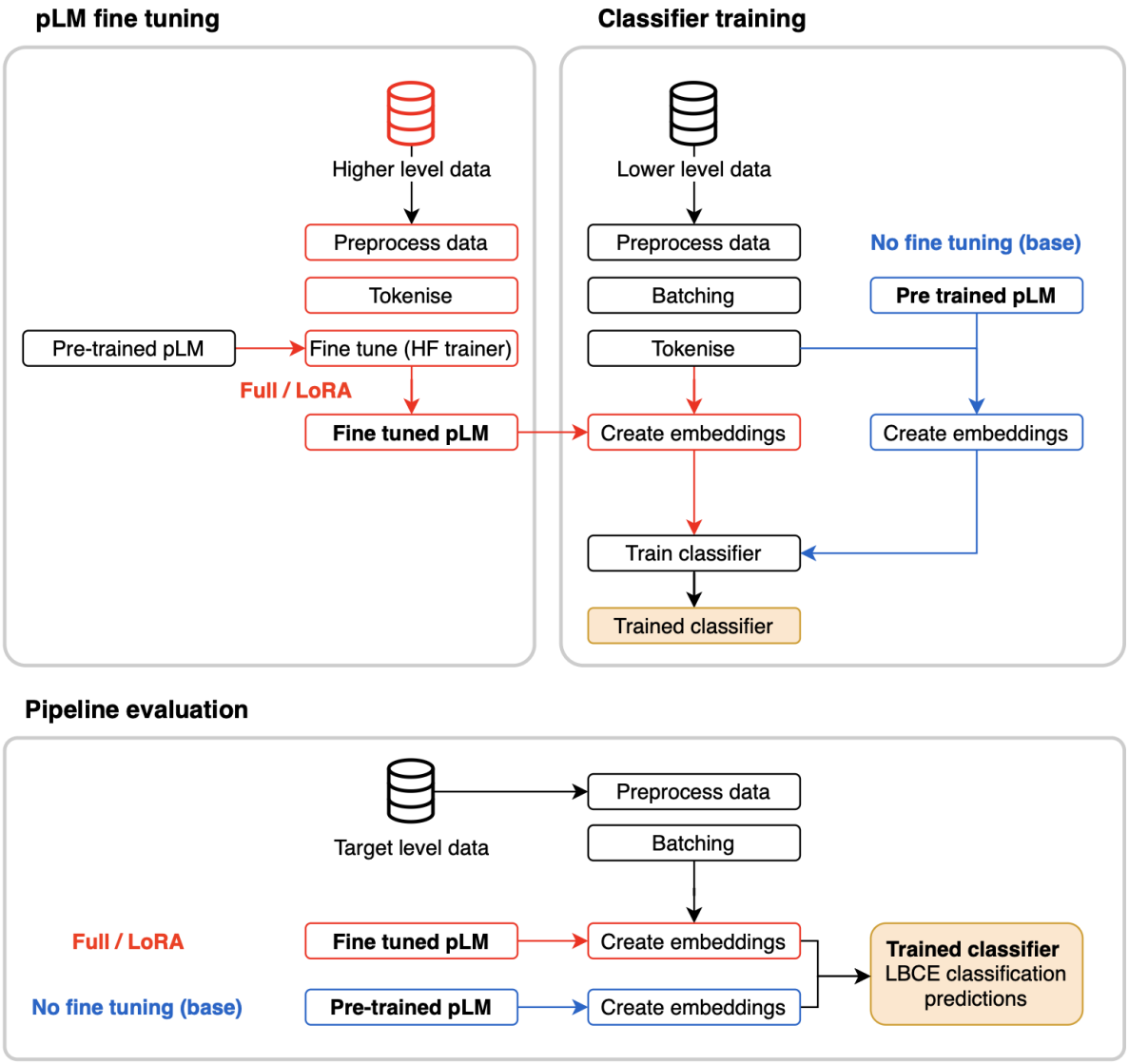


Figure 3.1: Overview of pipeline training and evaluation within for classifying embeddings from pre-trained or fine-tuned pLMs for the purpose of LBCE classification prediction. pLM fine-tuning modes with full fine-tuning or LoRA fine-tuning is shown in red. The base mode using the pre-trained pLM only is shown in blue.

for loading the relevant fine-tuned models. Classifier training/evaluation and target prediction only was conducted to derive the additional outputs/metrics.

3.3.5 Pipeline deviations for ESM-2 650M using Isambard-AI Phase 2

For ESM-2 650M models only, files were saved locally on the Project's login node therefore S3 upload functionality was removed from the Python scripts.

3.3.6 Pipeline deviations for running ProtT5 model

The following changes were made to the pipeline to enable ProtT5 processing on Ismabard-AI Phase 2

- A `ProtT5ForResidueClassification` neural network class was created containing the ProtT5 encoder with modules for the 2 layer neural network `PerResidueClassifier` head as the trainable classification head for fine-tuning. For LoRA fine-tuning, `ProtT5ForResidueClassification` defined the `LoraConfig` as per Schmirler et al. [13].

- `preprocess_csv` inserted a white space between residues in accordance with the ProtT5 tokenizer requirement.

- Batch size was reduced to 4 throughout due to GPU vRAM limits.

- For `preprocess_higher_level` , max sequence length padding was removed and special token handling was addressed through padding the end CLS token label position with '-100'. Dynamic padding was added following the `preprocess_higher_level` function to reduce memory overhead, using the Hugging Face `DataCollatorForTokenClassification` function.

- `batch_create` was updated to match ProtT5 token handling and to call the encoder only for embedding generation since no 'last_hidden_state' parameter is available for ProtT5.

- Loading of models for classifier training and target-level embedding generation was updated to ensure LoRA adapters were loaded correctly.

3.4 Pfeature pipeline

Assessment of Pfeature representations as input to the classifier head used lower-level data only for training/evaluation, and target data for final per-residue classification assessment. Pfeature.zip was downloaded from GitHub [15]. This zip file contained all Python scripts and supporting reference tables to generate engineered features for a given amino acid sequence. To maintain compatibility with the software package a Conda virtual environment with Python version 3.10 with additional required libraries was used. Pfeature representations were generated at a labelled residue level only, using the `Info_window` sequence as context to every labelled residue. For each `Info_window` sequence, outputs for the following properties were generated and concatenated together to create a single representation of the residue: amino-acid composition (AAC - 20 features), atomic composition (ATC - 5 features), bond composition (BTC - 4 features), physicochemical properties (PCP - 19 features), Shannon-entropy (SE - 1 feature) and conjoint triad calculation (CTC - 343 features). These feature groups were chosen as a general representation of basic amino acid properties, chemical composition, physiochemical properties, sequence diversity, and some contextual information.

Since Pfeature only accepts FASTA files, each 'info_window' had to be first converted to a temporary FASTA file which was fed into Pfeature. The pipeline then largely followed the logic for the ESM-2 and ProtT5 pipelines for classifier training and cross-validation, target prediction and metric/output generation / saving, with the main exception that Tensor batching was included as part of the classifier training. Within the batching, shuffling was introduced so that each batch contained a different sample composition ahead of each training epoch. As no GPU instances were required, all training/inference was completed on Apple MPS and CPU. A small edit was made to the Pfeature.py file to prevent the script from searching through the entire local drive for lookup tables. This edit is included in the Pfeature.py file in the GitHub repository.

3.5 Data analysis

3.5.1 Assessment metrics

The metrics generated during cross validation evaluation and final target prediction were accuracy, precision, recall, F1, Matthews Correlation Coefficient (MCC) and Area Under the receiver operating characterisic (ROC) Curve (AUC). Metrics were calculated using SciKit-learn.

Accuracy provides a measure of the correctly identified positive epitope (true positives, TP) and non-epitopes (true negatives, TN) as a ratio of all predictions on a scale of 0 - 1 where 1 is perfect accuracy. Precision measures the proportion of correctly identified positive epitopes (TP) as a ratio of total identified positive epitopes (TP and false positives, FP) on a scale of 0 - 1 where 1 represents perfect precision. Recall measures the proportion of correctly identified positive epitopes (TP) as a ratio of the true positive epitopes and false negative epitopes (TP and false negatives, FN) on a scale of 0 - 1 where 1 is perfect recall. F1 is a composite measure of precision and recall, calculated as their harmonic mean, on a scale of 0 - 1 with the best score at 1. Matthews Correlation Coefficient (MCC) is a measure of the overall quality of the binary classifier and takes into account true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN) measured on a scale of -1 to 1 where -1 is an inverse prediction, 0 is a random prediction and 1 is a perfect prediction. It provides a balanced measure even if classes are highly unbalanced. Accuracy, precision and recall are reliable measure for balanced datasets, however shown in Table 3.2, datasets used within the project not consistently balanced. AUC is a calculation of the area under the ROC curve and is a measure of classifier discriminatory performance across all possible thresholds, where 0 indicates a predictive performance worse than chance, 0.5 is no better than random and 1 is perfect predictive performance [7].

As MCC considers unbalanced datasets and all four possible prediction categories, it was chosen as the primary metric to be reported. MCC values presented, unless otherwise specified, use a default threshold of 0.5, corresponding to the selection of the class with the highest probability via Pytorch `torch.argmax` and `pytorch.softmax` applied to the classifier logits. The formula for MCC is detailed below:

$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

In addition per residue true labels and probabilities (and predicted labels) were generated for manual plotting of ROC curves with AUC values. Although MCC was the primary measures, the other measures detailed above were collected for completeness.

3.5.2 Outputs

The pipelines yielded the following metrics/outputs, file suffixes included for reference:
pLM fine-tuning (full fine-tuning and LoRA PEFT only)

- Trainable parameters summary (n trainable, n total, % trainable of n total) - `trainable_parameters_summary.csv`
- Trainer cross-validation evaluation performance metrics per fold and epoch - `cv_validation_metrics.csv`
- Trainer cross-validation training performance metrics per fold and epoch - `-cv_train_metrics.csv`
- Trainer cross-validation evaluation process and progress metrics per epoch `-cv_eval_log.csv`
- Trainer final training performance metrics per epoch (train losses and accuracy only) - `final_training_log.csv`
- Trainer final training process and progress metrics for final epoch - `final_training_summary.csv`

pLM classifier training and evaluation

- Classifier training and cross-validation evaluation performance metrics per fold and epoch - `clf_validation_metrics_by_fold.csv`
- Mean classifier training and cross-validation evaluation performance metrics per epoch - `clf_validation_avg_metrics.csv`

- Classifier cross-validation true label, class probability and predicted label per residue per fold for the highest mean AUC epoch from cross-validation - `[clf]_cv_roc_epoch.csv`
- Final classifier training performance metrics (train losses and accuracy only) - `clf_final_trained_metrics_by_epoch.csv`

Pipeline testing using target pathogen

- Test prediction performance metrics - `test_predictions.csv`
- Test prediction true label, probability and prediction per residue - `test_roc.csv`
- Confusion matrix - `test_confusion_matrix.csv`

3.5.3 Evaluation

The following subsection details the statistical methods and testing used. A statistical significance level of 0.05 was used throughout. Analysis was implemented in Python using the statsmodel library.

Effect of fine-tuning mode and pLM size

A linear model was fitted to assess the effects of fine-tuning mode and pLM size on target prediction MCC, with pathogen included as a blocking factor to account for heterogeneity between datasets. Pfeature results were excluded because they had no corresponding mode or model-size attributes. Statistical significance was assessed using a type II ANOVA with a significance level of 0.05. Pairwise comparisons between fine-tuning modes were performed using separate linear models with pathogen as a blocking factor, followed by type II ANOVA. The three pairwise comparisons were adjusted for multiple testing using statsmodels' default Holm–Šidák (HS) method. This was carried out to control for the the probability of false positives across the three pairwise tests (base vs LoRA, base vs full, LoRA vs full) at the 0.05 significance level.

Pathogen specific effects of fine-tuning mode and pLM size

To assess pathogen specific effects of fine-tuning mode and model size, linear models were applied to each pathogen mode and statistical significance determined with a type II ANOVA. To control for the expected proportion of false discoveries within the pathogen-specific tests, the Benjamini/Hochberg method (false discovery rate (FDR) correction) was applied at a significance level of < 0.05 separately across the pathogen-specific tests per mode and model size. For pathogens where mode was a significant effect, comparisons between modes were completed by fitting a linear model and performing pairwise comparisons. The p-values from the pairwise testing were adjusted using the default HS method to account for the probability of false positives from multiple pairwise corrections per pathogen at the 0.05 significance level.

Effects of model/representation type

To assess the effect of pLM or Pfeature representation types, a linear model was fitted to MCC with pLM type or representation as an effect, and pathogen as a blocking factor, with significance determined with a type II ANOVA. Pathogen-specific analysis was performed by fitting a linear model to model type or representation for each Pathogen followed by a type II ANOVA with FDR correction at a significance level of < 0.05, to control for the expected proportion of false discoveries. Pairwise comparisons between pLM/representation types were performed separately for each pathogen with FDR correction applied to correct for multiple pairwise comparisons within each pathogen.

Chapter 4

Results

4.0.1 Downstream target prediction

To test the final performance of each pipeline, per residue classification predictions were made for each pathogen dataset using the target-level data. Table 4.1 details the target prediction MCC scores for each pathogen, model or representation type, split by model size and fine-tuning mode where applicable. Box plots of the mean aggregated MCC scores across pathogen are presented in Figure 4.1. Target prediction AUC scores are included in Table A.1. The boxplots show that the median of MCC means across model

pLM/pFeature	ESM-2												ProtT5-XL			Pfeature
Parameters	8M			35M			150M			650M			1.2B			Pfeature
Mode	base	LoRA	full	base	LoRA	full	base	LoRA	full	base	LoRA	full	base	LoRA	full	Pfeature
Bunyaviricetes	-0.08	0.07	0.06	-0.07	0.03	0.15	-0.07	0.02	**0.20**	-0.07	-0.09	0.09	–	–	–	-0.06
Campylobacter jejuni	-0.21	-0.25	0.00	-0.22	-0.23	-0.28	-0.26	-0.32	0.00	-0.19	-0.26	-0.31	–	–	–	**0.06**
Clostridioides difficile	-0.05	-0.07	0.00	-0.04	-0.07	0.03	-0.00	-0.02	0.03	-0.08	-0.02	-0.07	–	–	–	**0.11**
Dengue virus	0.09	0.09	0.04	0.09	0.08	0.01	0.10	0.08	-0.01	0.11	0.11	0.04	–	–	–	**0.12**
Legionellales	0.21	0.12	-0.15	0.11	0.21	**0.30**	0.20	0.13	0.00	0.08	0.09	-0.17	–	–	–	0.24
Mycobacterium leprae	-0.06	-0.13	0.00	-0.14	-0.17	0.02	-0.05	0.06	0.13	-0.10	-0.07	0.00	–	–	–	**0.19**
Mycoplasmoidaceae	-0.28	-0.40	0.00	-0.40	-0.41	**0.25**	-0.27	-0.55	0.00	-0.34	-0.14	0.00	–	–	–	-0.11
Neisseria gonorrhoeae	-0.10	-0.23	0.00	-0.14	-0.23	0.00	-0.16	-0.14	-0.33	-0.19	0.00	0.00	–	–	–	**0.05**
Orthoparamyxovirinae	-0.02	0.03	0.16	-0.00	-0.22	-0.35	-0.14	-0.08	-0.04	**0.28**	0.11	-0.08	–	–	–	0.02
Orthopoxvirus	0.14	0.17	0.13	0.36	-0.11	**0.38**	0.24	0.22	0.29	0.25	0.17	0.38	0.16	0.15	0.00	0.23
Pasteurellaceae	-0.24	-0.33	0.00	-0.09	-0.24	-0.13	-0.16	-0.24	-0.01	-0.31	-0.19	**0.28**	–	–	–	-0.02
Yersinia pestis	0.25	0.17	-0.20	0.25	**0.32**	0.17	0.27	0.24	0.00	-0.10	-0.14	-0.24	–	–	–	0.08

Table 4.1: MCC performance across ESM-2, ProtT5-XL, and Pfeature models. The highest MCC for each pathogen is shown in bold.

types was typically around 0, indicating that the average discriminatory performance of the pipelines to correctly classify labelled residues was limited. Boxplots for the ESM-2 checkpoints also had wide 95% confidence intervals, indicating increased uncertainty in average predictive performance in MCC between pathogens. Target prediction MCC scores for each pathogen by model, fine-tuning mode and model size, along with Pfeature data is presented in Figure 4.2. This data showed that there were some pathogen-specific trends with fine-tuning mode and pLM model size, however these trends were not directionally consistent. Moreover certain pathogens showed no trends with fine-tuning mode and model size and worse than random discriminatory performance.

Pfeature representation pipelines delivered the highest MCC values for pathogens Campylobacter jejuni, Clostridioides difficile, Dengue virus, Mycobacterium leprae and Neisseria gonorrhoeae. However these MCC values were lower than the highest MCC values for all other pathogens using the pLM-derived pipelines. This suggests that although Pfeature provided competitive and superior performance for certain pathogens, pLMs were able to derive higher MCC scores with certain pathogen/mode and model size configurations at the cost of increased variability.

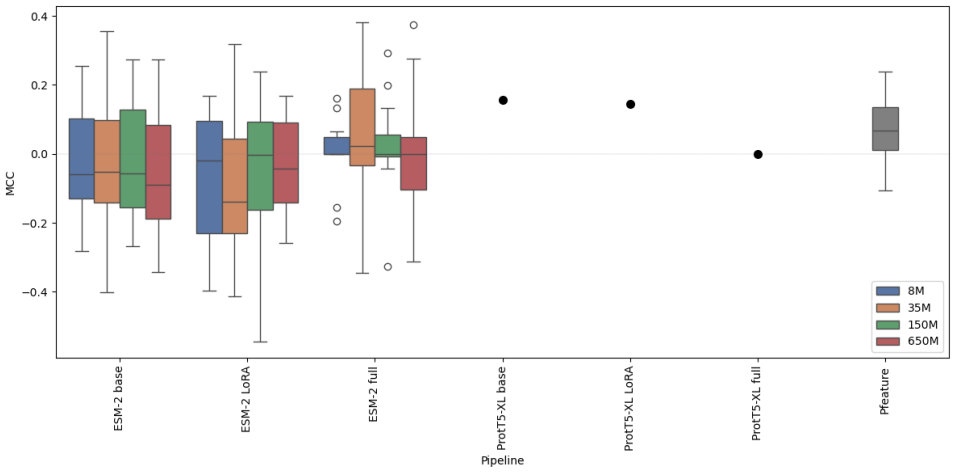


Figure 4.1: Target prediction aggregated MCC values across the three fine-tuning modes and Pfeature representations for all 12 pathogens

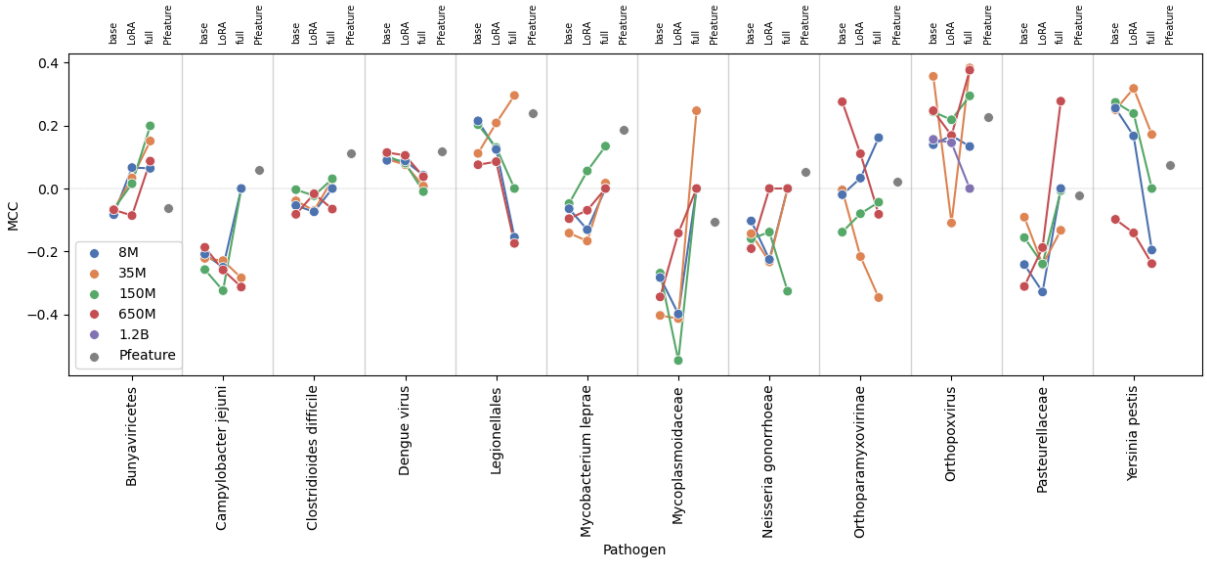


Figure 4.2: Target prediction MCC values by Pathogen across the three fine-tuning modes and Pfeaure representations. Parameter sizes ESM-2: 8M, 35M, 150M and 650M, ProtT5-XL: 1.2B

Comparison by fine-tuning mode and model size

An objective of this project was to compare the performance of fine-tuning modes and model sizes using different pLMs for the transfer learning task of LBCE prediction. To statistically analyse the downstream target prediction results, an ANOVA was applied to a linear model for fine-tuning mode and model size effects on MCC, with pathogen as a blocking factor to control for heterogeneity across pathogen datasets. This analysis identified that fine-tuning mode had a significant overall effect on MCC ($p = 0.04$), whereas model size did not ($p = 0.61$). Pathogen also had a significant effect ($p < 0.001$); Table 4.2. From an initial pairwise comparison between mode pairs, the effect of LoRA vs full-fine-tuning was significant ($p = 0.026$). However, following Holm–Šidák correction across the three pairwise comparisons (base vs LoRA, base vs full, LoRA vs full) no mode pairs showed statistically significant effects (Figure 4.3). This therefore indicated that when controlling for pathogen and correcting for the probability of false positives in pairwise comparisons, there was no effect of fine-tuning mode.

As the effect of pathogen was found to be significant in the blocked ANOVA ($p < 0.001$), a pathogen-specific analysis was performed which identified significant mode effects. FDR corrected ANOVA results for the individual pathogen models highlighted that fine-tuning mode was a significant effect for several pathogens: Bunyaviricetes ($p = 0.03$), Dengue virus ($p < 0.001$), Mycobacterium leprae ($p = 0.03$),

Table 4.2: ANOVA results for ESM-2 and ProtT5-XL, blocking pathogen.

Term	Sum of Squares	df	F	p-value
Mode	0.126	2	3.291	0.040
Model size	0.039	4	0.507	0.730
Pathogen	2.262	11	10.783	< 0.001
Residual	2.460	129	–	–

Table 4.3: Pairwise mode comparisons for ESM-2 and ProtT5-XL, blocking for pathogen and with Holm–Šidák correction.

Comparison	F	p	p_{HS}	Significant
Base vs LoRA	1.709	0.195	0.242	No
Base vs Full	2.345	0.130	0.242	No
LoRA vs Full	5.164	0.026	0.075	No

Mycoplasmoidaceae (p = 0.03) and Yersinia pestis (p = 0.04) (Table A.2). Model size was not significant for any pathogens.

For these pathogens, pairwise comparisons found that the effect of full fine-tuning was significant for all five pathogens after HS correction (Table A.3). The effect of no fine-tuning vs LoRA was not significant in any cases. In addition, the directionality of the mode effect was pathogen dependant. For example, the full fine-tuned configurations for Bunyaviricetes and Mycoplasmoidaceae showed improved performance compared to the base and LoRA fine-tuning modes. For example, when compared to both base and LoRA fine-tuning modes, the MCC coefficients for the pairwise comparisons for Bunyaviricetes full vs LoRA, 0.118 (p = 0.032), full vs base, 0.196 (p = 0.005) and Mycoplasmoidaceae full vs LoRA, 0.0436 (p = 0.019), full vs base (p = 0.019). Conversely, for Dengue Virus and Yersinia Pestis, the MCC coefficients for the pairwise comparisons showed worse predictive performance for the fine-tuned models. For Dengue virus, the full vs LoRA pairwise comparison coefficient was -0.068 (p = 0.001), base vs LoRA 0.080 (p = 0.001) and for Yersinia Pestis, full vs LoRA was -0.211 (p = 0.027), full vs base -0.236 (p = 0.025) (Figure 4.3. Due to these opposing findings, analysis to understand the potential causes in diverging predictive performance for some of these pathogens is detailed in subsequent sections below.

Comparison of model/representation type

To compare the performance of the different pLM or Pfeature representation types, a linear model was fitted to MCC with pLM type or representation as an effect and pathogen as a blocking factor. An ANOVA showed that the effect of pLM type or representation was significant when controlling for pathogen (p = 0.022) (Table 4.4). Pathogen-specific analysis was performed which showed no significant overall effect of model/representation after FDR correction (Table A.4). Pairwise comparisons of model/representation types showed that Pfeature had significantly higher MCC than ESM-2 for Campylobacter jejuni (+0.270, FDR-adjusted p = 0.033), Clostridioides difficile (+0.142, FDR-adjusted p = 0.005), and Mycobacterium leprae (+0.229, FDR-adjusted p = 0.030) (Table A.5). As both pLMs encompassed several fine-tuning modes, as well as ESM-2 predictions having 4 model checkpoints per fine-tuning mode, this analysis did not consider these permutations per pLM test.

Table 4.4: ANOVA results for Pfeature, ESM-2, and ProtT5-XL, blocking pathogen.

Term	Sum of Squares	df	F	p-value
Model	0.145	2	3.904	0.022
Pathogen	2.286	11	11.171	< 0.001
Residual	2.698	145	–	–

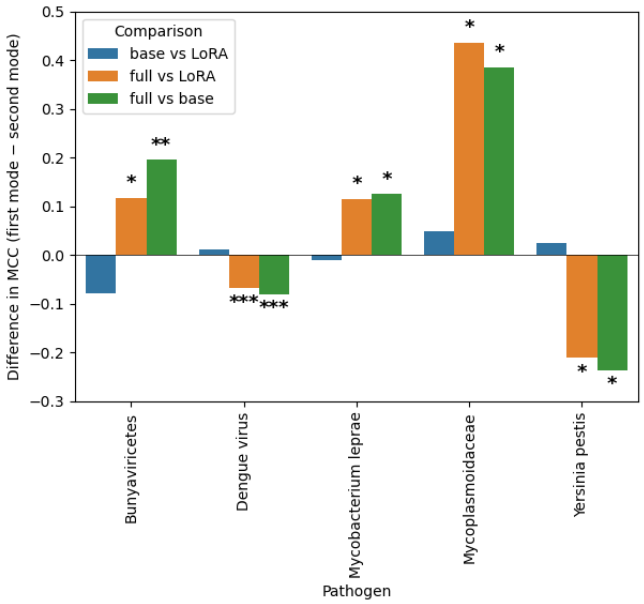


Figure 4.3: Pairwise differences in target prediction MCC by mode for pathogens with significant differences using Holm–Šidák-adjusted p-values ($p < 0.05$). Asterisks represent the significance level of the pairwise differences: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$.

4.0.2 Pathogen case studies

To identify potential drivers for differences in target prediction performance, Bunyaviricetes and Dengue virus were chosen as representative cases for opposing statistically significant effects for full fine-tuning on MCC.

pLM fine-tuning

Model fine-tuning performance was evaluated using 3 folds from the higher-level data set separated into training and validation splits. For each fold, fine-tuning was performed over 3 epochs. Mean results for the folds across epochs is detailed in Table 4.5. For Bunyaviricetes, the evaluation metrics showed

Table 4.5: pLM fine-tuning Trainer evaluation metrics for LoRA and full fine-tuning across epochs for Bunyaviricetes and Dengue virus.

Pathogen	Mode	Epoch	Eval Loss	Eval Accuracy	Eval MCC
Bunyaviricetes	LoRA	1.0	0.40	0.82	0.46
		2.0	0.39	0.82	0.45
		3.0	0.39	0.82	0.48
	Full	1.0	0.46	0.75	0.16
		2.0	0.40	0.80	0.50
		3.0	0.38	0.80	0.50
Dengue virus	LoRA	1.0	0.69	0.61	0.11
		2.0	0.69	0.64	0.13
		3.0	0.68	0.64	0.16
	Full	1.0	0.63	0.65	0.31
		2.0	0.62	0.69	0.33
		3.0	0.64	0.71	0.35

that full fine-tuning improved MCC, with an associated expected training behaviour, particularly for the larger parameter models. For example, between epoch 1 and 3 for the full fine-tuned 150M parameter

model, mean loss reduced from 0.16 to 0.50, indicating training was optimising the model weights to minimise the training objective, mean accuracy increased from 0.75 to 0.8, indicating that the model was learning to accurately predict the correct class, and mean MCC increased from 0.35 to 0.585, showing training improvement across all prediction categories. For Dengue Virus, however, the improvement in MCC during marginally increasing from 0.31 to 0.35 between epochs 1 and 3.

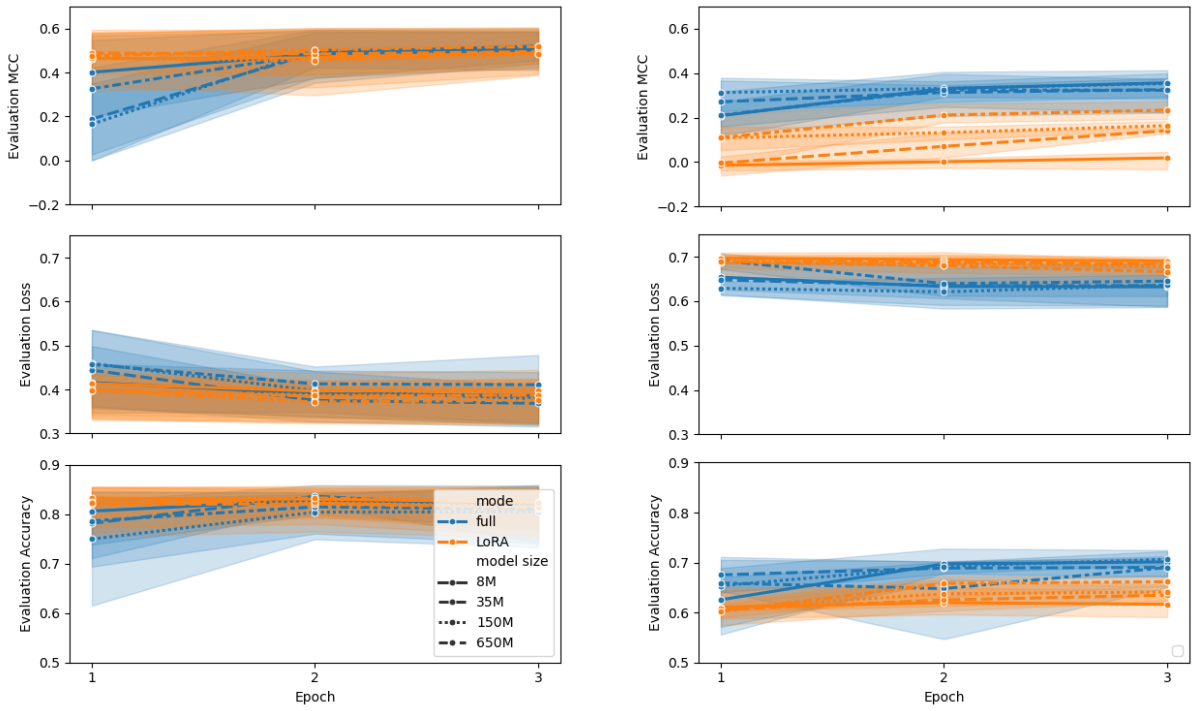


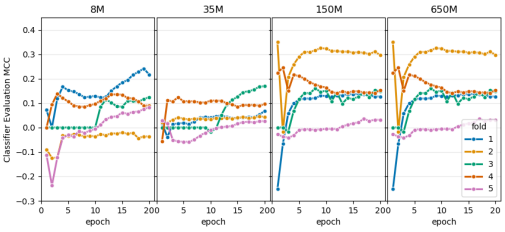
Figure 4.4: Trainer cross validation evaluation MCC, accuracy and loss for Bunyaviricetes (left) and Dengue virus (right) from the evaluation metrics for model evaluation after each training fold. Confidence intervals are 95% confidence interval for the 3 validation folds.

Classier training and evaluation

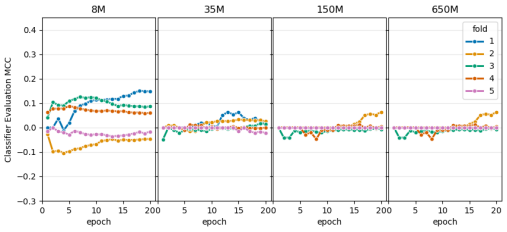
Next, evaluation metrics from the `PerResidueClassifier` classification head trained using the lower-level data were explored. Csv files containing metrics by epoch are saved in the GitHub repository due to the table sizes. Bunyaviricetes demonstrated improved classifier training dynamics as evidenced by the MCC, loss and accuracy data by epoch of the hold-out folds compared to Dengue virus. For full fine-tuning, the MCC for Bunyaviricetes was higher on average across epochs, as well as having higher average accuracy and lower average losses (Figure 4.5), trending with model size. Dengue virus showed the opposite effect, with base embeddings showing improved MCC across folds with a reverse trend with model size. In addition, the MCC for the larger, full-fine tuned models had collapsed to 0 for several of the hold-out folds, whereby the classifier had collapsed to predicting one class during training. These trends were reversed comparing base models between the two pathogens.

Both pathogens showed high variation in MCC, loss and accuracy across modes and model sizes between validation folds. Therefore to evaluate the classifier discrimination performance across folds independently of the fixed classification threshold used in MCC, receiver operating characteristic (ROC) curves were created for the highest mean AUC epoch using the per-residue classifier prediction scores and true labels for unmasked residues. The ROC curves with AUC scores for the 150M parameter ESM-2 models using full fine-tuned pLM embeddings for Bunyaviricetes and Dengue virus are shown in Figure 4.6 and the corresponding curves using base pLM embeddings in Figure A.1.

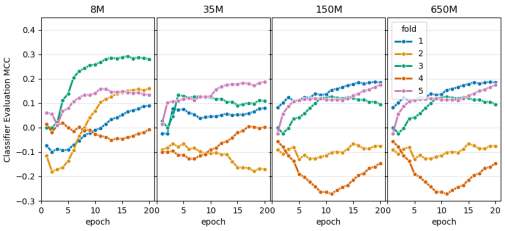
For the Bunyaviricetes full fine-tuned embeddings, AUC values varied between folds (0.498 to 0.848), with fold 3 having the best discriminatory performance (AUC = 0.848) and fold 1 having performance on par with a random classifier (AUC = 0.494). Since there was substantial variability in discriminatory performance for the classifier across folds, this suggested fold-specific classifier sensitivity differences. Therefore to investigate whether class imbalance was responsible for the per fold AUC results, the number



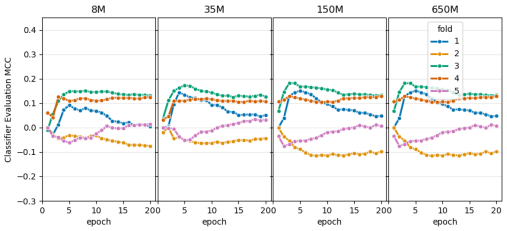
(a) Bunyaviricetes full-fine tuned embeddings



(b) Dengue virus full-fine tuned embeddings

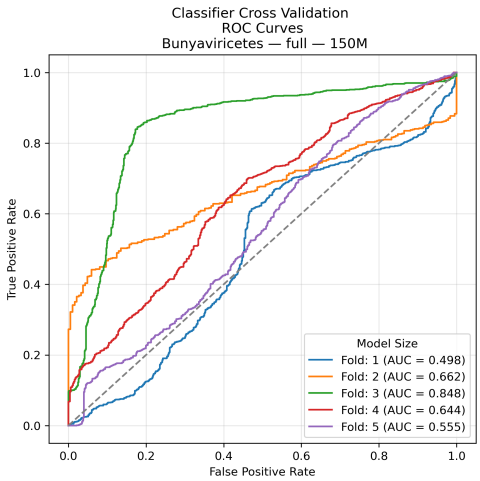


(c) Bunyaviricetes base model embeddings



(d) Dengue virus base model embeddings

Figure 4.5: Classifier evaluation MCC by epoch per fold for Bunyaviricetes and Dengue virus embeddings.



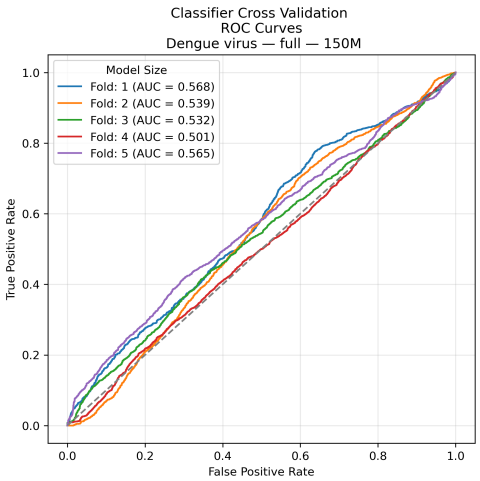


Figure 4.6: Classifier cross validation evaluation ROC curves from each validation fold for the 150M ESM-2 model using full fine-tuned pLM embeddings. Bunyaviricetes (left) and Dengue virus (right).

Table 4.6: Number and percentage of labelled residues per hold-out fold for classifier evaluation.

Pathogen	Hold-out fold	Positive (n)	Negative (n)	Positive (%)	Negative (%)
Bunyaviricetes	1	1057	666	61.3	38.7
	2	659	185	78.1	21.9
	3	1330	840	61.3	38.7
	4	1126	499	69.3	30.7
	5	1144	601	65.6	34.4
Dengue Virus	1	1269	1044	54.9	45.1
	2	1061	2646	28.6	71.4
	3	2403	1292	65.0	35.0
	4	3685	1462	71.6	28.4
	5	1271	1251	50.4	49.6

and percentage of unmasked positive and negative epitope residues per fold was calculated and reported in Tables 4.6.

However the weightings showed that folds 1 and 3 for Bunyaviricetes had identical epitope and non-epitope weightings (61.3% epitope and 38.7% non-epitope) despite the contrasting AUC values, indicating that the variation in evaluated classifier performance was not due to differences in class weighting alone. Additionally, Dengue virus which had comparatively less variation in AUC between folds (0.501 - 0.568) for the equivalent model size and mode, had substantially more variation in class weighting across folds, suggesting that class weighting again was unlikely to contribute to the lack of classifier discriminative ability across folds. Although this check used the labelled data from the original lower-level dataset a more robust check would have been to confirm the weightings after preprocessing due to the `sliding_window` function.

Bunyaviricetes & Dengue virus MCC threshold testing

To investigate whether classification performance was being affected by the decision threshold on the negative and positive classes, MCC was evaluated across the full range of classification thresholds (0-1) for Bunyaviricetes and Dengue virus 150M ESM-2 models for the base and fine-tuned embedders.

The threshold analysis showed that MCC was heavily threshold dependant, particularly for the classifier evaluation during training using the lower-level data. For Bunyaviricetes, the maximum MCC was obtained at 0.73, producing an MCC score of 0.22, compared with 0.15 using the default threshold of 0.5 (Figure 4.7). For the equivalent Dengue virus configuration, the maximum MCC was obtained at a threshold of 0.6, increasing MCC from 0.01 at the default threshold to 0.05. Dengue virus threshold response showed poor robustness when compared to Bunyaviricetes. Small deviations from of the best threshold resulted in a rapid decrease in MCC to negative MCC values, with MCC values of 0 at a threshold of 0.4 and below. Conversely Bunyaviricetes had a more robust MCC threshold range around the optimal threshold and was associated with positive MCC values. This indicated that Bunyaviricetes provided a stronger and more robust separation between classes.

To test whether these thresholds returned improved target predictions, the best threshold values were applied to the target prediction per residue predictions. Using the new threshold values, the target-level MCC for Bunyaviricetes reduced from 0.19 to 0.08. For Dengue virus, the target-level MCC value marginally improved from to -0.01 to 0.04. This therefore showed that optimisation of classifier threshold did not consistently improve target-level MCC performance for this small sample. These results also suggest that the probability-score distributions for the classifier during training evaluation using the lower-data and inference with target-level data differ. Consequently, a threshold boundary that offers better separation for embeddings using lower-level data may not be well-suited for classifying embeddings from the target-level data. Therefore to test this, the same threshold testing was carried out for the test results. Both curves are plotted together in Figure 4.7. Due to the different MCC threshold curves for the lower-level data and target-level data, and diverging optimal threshold values observed to different magnitudes for both pathogens, these result suggests the transfer of classification thresholds between lower-level and target datasets is pathogen dependant.

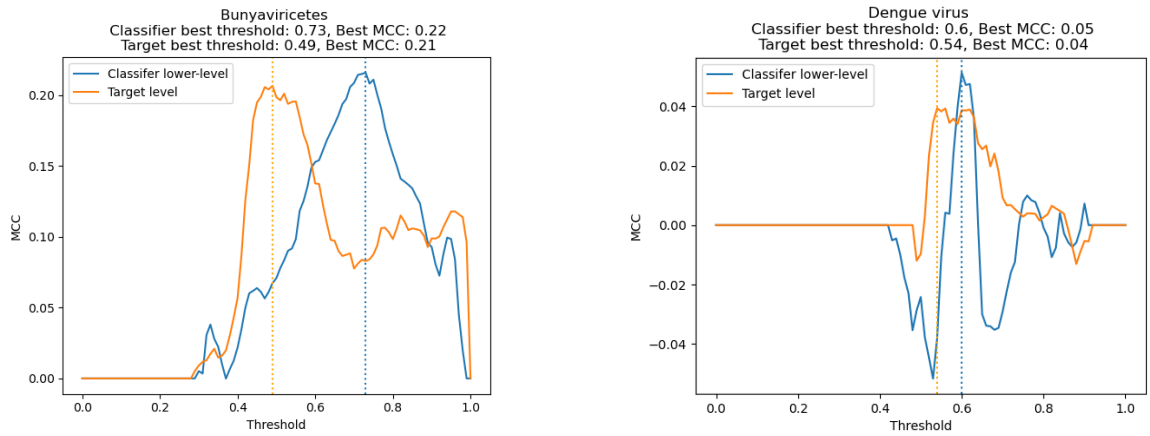


Figure 4.7: MCC threshold testing on classifier evaluation and target prediction data for Bunyaviricetes and Dengue virus, 150M full fine-tuned embedder.

4.0.3 Pfeature: comparison of classifier evaluation dynamics

As the threshold-independent classifier evaluation performance (ROC-AUC) was highly variable between folds for the Bunyaviricetes 150M full fine tuned embedding configuration, with less fold-fold variability observed for the equivalent Dengue virus configuration, fold-level ROC-AUC curves were generated for the Pfeature classifier evaluation using the same pathogens (Figure 4.8). With Pfeature, there was less AUC fold to fold variation for Bunyaviricetes (0.511 - 0.648) compared to the ESM-2 150M full fine-tuned configuration (0.494 to 0.848). For Dengue virus, Pfeature exhibited slightly more variation in AUC (0.465 to 0.588) compared to the ESM-2 150M full fine-tuned configuration (0.501 to 0.568). These results indicate that the ability of the classifiers to discriminate the two classes differed between the representation approaches and pathogens. In particular, Bunyaviricetes showed substantial variability between folds which was not seen using Pfeature, whereas Dengue virus was more comparable to Pfeature. Given the Bunyaviricetes results, this suggests that ESM-2 may encode more information for the classifier to discriminate classes, however this information was not able to be leveraged consistently across all folds.

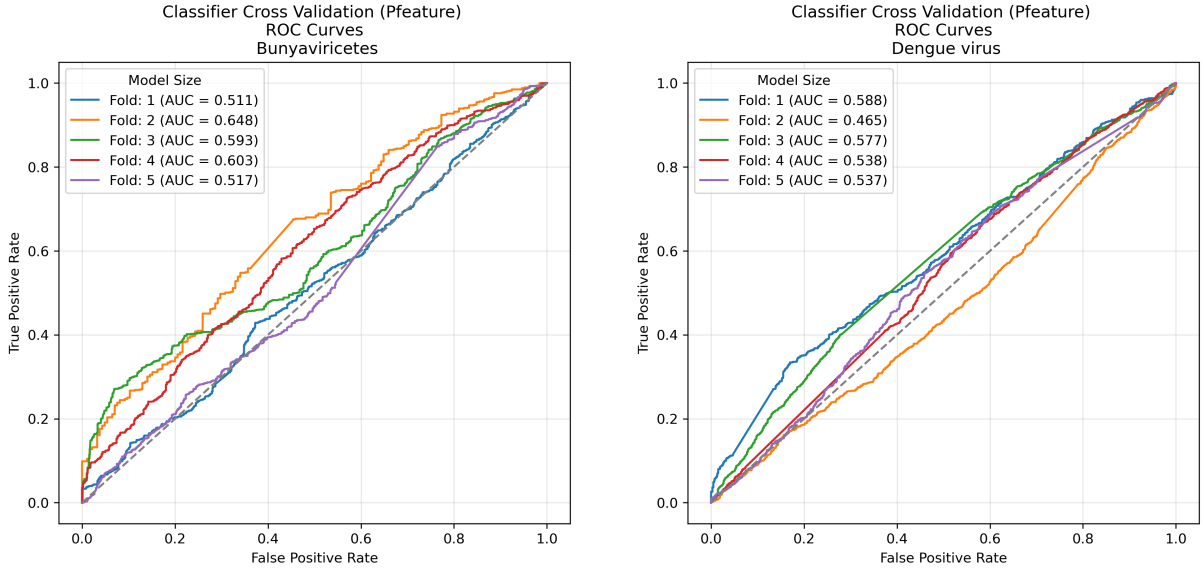


Figure 4.8: Classifier cross validation evaluation ROC curves from each validation fold for the Pfeature representation pipeline. Bunyaviricetes (left) and Dengue virus (right).

4.0.4 Orthopoxvirus: ProtT5-XL

The fine-tuning evaluation metrics for loss, accuracy and MCC for ProtT5-XL fine-tuning using the Orthopoxvirus dataset higher-level data exhibited improved training dynamics and performance for LoRA compared to full fine-tuning (Table 4.7). This implies that with full fine-tuning that the pLM's weights were not being sufficiently adapted to the downstream task, possibly through non-optimal hyperparameter settings such as the learning rate, which was the same for both LoRA and full fine-tuning. Furthermore, the pLM's pre-trained weights may have provided sufficient representative capacity applicable to the task that full fine-tuning negatively impacted through excessive changes related to the higher-level data and within the available fine-tuning time.

Table 4.7: Trainer metrics for fine-tuning ProtT5-XL using Orthopoxvirus higher-level data

Pathogen	Mode	Epoch	Evaluation loss	Evaluation accuracy	Evaluation MCC
Orthopoxvirus	LoRA	1	0.44	0.80	0.51
		2	0.42	0.80	0.53
		3	0.41	0.80	0.54
	full	1	0.61	0.62	0.21
		2	0.53	0.75	0.44
		3	0.49	0.76	0.48

The improved fine-tuning performance for the LoRA fine-tuned ProtT5-XL pLM translated into both improved classifier evaluation performance and downstream target prediction compared to the full fine-tuned pLM. The mean training evaluation MCC for the base model was 0.30 for the base model, 0.28 for LoRA and -0.11 for full-fine tuned embeddings (data saved in the GitHub repository). Target prediction MCC for the base model was 0.16, 0.15 for LoRA and 0 for the full fine-tuned mode. In comparison, the best target prediction MCC value for ESM-2 was 0.38 for the full fine tuned 35M parameter model and 0.23 for Pfeature.

As the ProtT5-XL full fine-tuned model had collapsed to predicting one class for the target-level data, MCC threshold testing was carried out to determine whether altering the threshold would impact these results (Figure 4.9). For the full fine-tuned embeddings, the optimal target-level MCC threshold for the classifier evaluation was 0.67 with an MCC of 0.08, whereas the optimal target-level threshold was at 0.89 with an MCC value of 0.03. Additionally the MCC values across the threshold range were either 0 or negative with a small peak at 0.89 for the maximum MCC value. In comparison, LoRA and base pLM embeddings showed relatively similar MCC threshold testing results. With the LoRA pLM embeddings, the optimal threshold for the classifier evaluation was 0.53 with an associated MCC of 0.31 and a target-level threshold of 0.85, MCC 0.31. For the base pLM the optimal threshold for the classifier evaluation was 0.48, MCC 0.30 and a target-level threshold of 0.71, MCC 0.25. These results suggest that full fine-tuning of the ProtT5-XL pLM may have resulted in representations that were highly adapted to the higher-level data and did not generalise effectively to the lower-level data during classifier training or the target-level data. Conversely, the comparatively limited task adaptation from LoRA fine-tuning may have allowed the pLM to retain information from the pLM's pre-trained weights that were useful to the classifier. This interpretation is supported by the similar MCC threshold profiles and associated MCC scores for the base and LoRA modes, suggesting that the embeddings generated by the pLM in both instances retained increased classification utility.

4.0.5 LoRA - number of trainable parameters

The number of trainable parameters with LoRA as a percentage of the total parameters is shown in 4.8 for each pLM. Since the same LoRA configurations were used for each model as per the parameter settings detailed by [13], the proportion of trainable parameters for each model checkpoint was not consistent. The proportion of trainable parameters decreased from 2.1% for the 8M ESM-2 model, to 0.5% for the 650M ESM-2 mode - an approximate 4-fold decrease. Consequently comparisons of model sizes for LoRA test runs are potentially confounded by differences in the percentage of trainable parameters, with larger models not having the same degree of adaptation from LoRA matrices compared to the smaller models.

4.0.6 Results summary

The key results are summarised below.

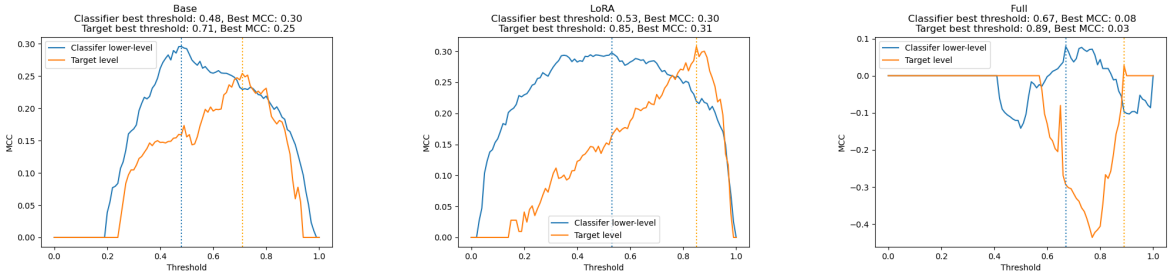


Figure 4.9: MCC threshold testing on classifier evaluation and target prediction data for Orthopoxvirus using ProtT5-XL.

Table 4.8: Comparison of protein language models (pLMs), parameter counts, and percentage of trainable parameters.

pLM	**Parameters (n)**	**% Trainable**
ESM-2	8M	2.1%
ESM-2	35M	1.4%
ESM-2	150M	1.1%
ESM-2	650M	0.5%
ProtT5-XL	1.5B	0.2%

- The average performance of pLM and Pfeature-derived representation pipelines for per-residue LBCE prediction performance was limited.
- The Pfeature-derived pipeline produced the highest MCC performance for 5 / 12 pathogens. The pLM pipelines however produced higher maximum MCC scores for the remaining 7 pathogens at the cost of increased variability between modes and model sizes.
- Model/representation type had a significant effect on MCC when controlling for pathogen, although no significant effects remained after FDR correction in the pathogen-specific analyses. Pairwise analysis showed that Pfeature significantly outperformed ESM-2 for 3 pathogens.
- The effect of pLM fine-tuning mode or model size on MCC was not statistically significant when controlling for pathogen.
- Pathogen-specific fine-tuning mode effects were however observed for 5 pathogens.
  - For all 5 pathogens, the effect of full fine-tuning was statistically significant compared to no fine-tuning and LoRA fine-tuning. LoRA fine-tuning had no effect compared to no fine-tuning.
  - The directionality of this effect was pathogen dependant. Fine-tuning positively impacted predictive performance for 3 pathogens and was detrimental for the remaining 2 pathogens.

Bunyaviricetes (positive impact of fine-tuning) and Dengue virus (negative impact of fine-tuning) ESM-2 case studies:

- pLM fine-tuning using higher-level data improved fine-tuning MCC and training dynamics for Bunyaviricetes when compared to Dengue virus.
- For Bunyaviricetes, classifier cross-validation had higher MCC values on average for the full fine-tuned pipeline, trending with model size when compared to the base Bunyaviricetes model.
- Both pathogens exhibited high fold variability during classifier cross-validation, except for the full fine-tuned pipeline for Dengue virus which showed collapse to predicting one class only for several folds.
- Within-fold class weighting was unlikely to have contributed to the variable per-fold discriminative ability of the classifier. This was demonstrated through ROC-AUC and class weighting analysis of the classifier cross-validation data.

- MCC threshold testing indicated differences in classification thresholds between lower-level and target-level data. The magnitude of these differences was pathogen dependant. Transferring the optimal threshold for the lower-level for target inference did not translate to improved target-level classification discrimination.

Pfeature: comparison of classifier evaluation dynamics

- ROC-AUC results from classifier cross-validation with Bunyaviricetes Pfeature representations were less variable than those from the ESM-2 150M full fine-tuned configuration.
- This suggests that ESM-2 may encode more information for the classifier to confidently discriminate classes compared to the Pfeature representations. However this information was not consistently leveraged across all folds.

Orthopoxvirus: ProtT5-XL

- LoRA fine-tuning of ProtT5-XL using higher-level data improved fine-tuning MCC and training dynamics compared to full fine-tuning.
- The LoRA fine-tuning performance translated into superior classifier evaluation performance and downstream target prediction compared to the full fine-tuning.
- MCC threshold testing of the classifier and target prediction results suggested full fine-tuning negatively impacted the ability of the classifier to discriminate epitope classes.

4.0.7 Discussion

In contradiction to Schmirler et al. [13], this project did not find an average downstream target prediction performance improvement with task-specific pLM fine-tuning. Instead this project identified fine-tuning effects that were pathogen-specific and directionally inconsistent with fine-tuning mode and model size. Although Schmirler et al. [13] did not include any epitope prediction tasks, they tested several per residue prediction tasks, including secondary structure and protein disorder. Secondary structure in particular was a per-residue 3-class prediction task to predict whether a residue is part of a protein's alpha-helix, strand or other/non-regular. They found insignificant gains with fine-tuning for ESM-2 and ProtT5 and marginally worse performance with Ankh models. Given the similarity in relation to the task of classification only, rather than the domain of the task, this could be considered as a result that supports the findings within this project, i.e. insignificant average gains with fine-tuning. With disorder prediction, the authors found performance gains for the ESM-2 35M and 150M models. Interestingly, the ESM-2 150M full fine tuned model outperformed the larger 650M ESM-2 model, an observation that was identified for several proteins in this project's LBCE prediction task. Ankh models again showed poorer performance than the baseline. The authors attributed this to Ankh's pre-training procedure and architecture which was optimised using data that overlapped with some of the downstream tasks tested [13].

There were also implementation differences between [13] and this project. This project completed one iteration for each pLM or Pfeature configuration per pathogen, controlled through the same random seed initialisation. In [13] the authors tested each model-task combination multiple times using different random seeds and averaged the results, informing the repeatability and robustness of each model-task variation. Secondly, the Jupyter notebooks provided by [13] include the LoRA fine-tuning implementation without full transparency of the full fine-tuning implementation. As such, certain hyperparameters for LoRA fine-tuning, such as learning rate, adopted for both fine-tuning methods in this project, may not be adequately optimised for full fine-tuning. For example, Leite et al., [7] used a learning rate of 1e-5 when fine-tuning the ESM-2 650M model for per protein LBCE classification prediction, using the same phylogeny-aware dataset implementation leveraged by this project. This learning rate was significantly lower than the 3e-4 learning rate used by [13] and this project. Consequently the training update magnitudes were not equivalent between LoRA and full fine-tuning. LoRA uses a low-rank parametrisation of the weight updates resulting in smaller updates compared to full fine-tuning [12]. Consequently the learning rate may have been too aggressive for full fine-tuning leading to instances of observed class prediction imbalance. A more robust comparison would have been to optimise the learning rate for the full-fine-tuning to a learning rate with similar weight updates to that of LoRA (or vice versa). Despite this, even with the equivalent learning rate as [13], LoRA did not show consistent performance gains to the base, non-fine tuned embedder.

The pathogens tested by Leite et al. [7] included a range of taxonomically diverse pathogens, one of which, Orthopoxvirus, was tested in this project. The authors showed either AUC performance losses or insignificant gains for this pathogen using a fine-tuned ESM-2 model 650M pipeline with phylogeny-aware data compared to other workflows with epitope prediction pLM transfer learning. In addition, this project generated higher Orthopoxvirus MCC values for several of the ESM-2 configurations. For example the full fine-tuned ESM-2 650M model pipeline produced an MCC of 0.38 vs 0.17 in [7]. Again there were various implementation differences, namely [7] used transfer learning to develop a per-sequence classification pipeline using sequences of labelled peptides, whereas this project developed a per-residue classification task with sequences containing combinations of positively and negatively labelled epitope residues and unlabelled residues. Although both used a sliding window to ensure a maximum sequence length for ESM-2, [7] averaged the embeddings for any overlapping regions to provide consistent embedding information for these regions. Furthermore, the authors used a random forest classifier instead of a 2-layer neural network classifier. Hyperparameter optimisation was conducted for the random forest classifier which is likely to have improved classification performance. Data-scarce pathogens (including Orthopoxvirus) used the default random forest hyperparameters [7].

Domain-specific features generated using Pfeature produced better predictions to pLM pipelines, specifically ESM-2, for certain pathogens. However there were differences between these pipelines that should be considered. Firstly the Pfeature pipelines used labelled epitope residues, with some information for unlabelled residues incorporated through the 15 residue `info_window` context window. Consequently, the Pfeature representations seen by the classifier contained a much higher proportion of labelled information compared to the pLM pipelines. Although unlabelled residues were masked before calculating losses, the pLM embeddings contained information from unlabelled residues which could have impacted the quality of the representations used for pLM fine-tuning and classifier training. Additionally batch shuffling was employed as part of the Pfeature pipeline during classifier training, meaning samples were randomly allocated to each batch at the beginning of each epoch. This meant that the classifier saw different batch configurations which may have helped with the classifier's ability to generalise. For pLM pipelines, omitting this step may have meant that training was less robust to unseen samples, potentially by being partially adapted to the configuration of the training batches. As a note, the Hugging Face Trainer class used in for pLM fine-tuning includes batch shuffling as default.

Previous work has shown that applying per-residue epitope classification to single biochemical properties of amino acids, like those in Pfeature, did not provide sufficient information for classifying epitope and non-epitope LBCE residues [26]. However, when combining the biochemical properties along with other amino acid descriptors, and using this as input to a bidirectional long short-term memory (BiLSTM) neural network and multilayer perceptron classifier, the authors were able to generate per-residue LBCE prediction MCC scores on par with those using ESM-2 ahead of the BiLSTM and classifier: ESM-2: 0.14, biochemical properties + descriptors: 0.13 [26]. These MCC results could therefore support the challenge of the LBCE per-residue prediction task found within this project. The authors also discuss wider considerations, highlighting that the predictions using their best performing workflows could not replace experiments at this stage. One potential factor was the the quality of the experimental epitope data; epitope datasets contain data generated using heterogenous experimental methods and binding affinities, making reliable predictions a challenge [26].

Chapter 5

Conclusion

This project included the development, training, evaluation and final performance assessment of pipelines for the downstream task of LBCE per-residue classification prediction. This included embedding generation from two different pLMs (ESM-2 and ProtT5-XL), across different fine-tuning methods and model size checkpoints. Additionally, LBCE per-residue classification predictions employing engineered Pfeature representations for comparison to domain-specific features were assessed. The pipelines used a phylogeny-aware approach that controlled for data leakage due to potential sequence similarity from evolutionary related organisms. As stated previously, this approach has been successfully used by other authors to generate informed classification predictions for LBCE downstream predictive tasks.

Considerable effort was dedicated to the development of the different pipelines, ensuring pLM fine-tuning and classifier training with phylogeny-informed datasets. As part of this, appropriate metrics were generated to support process evaluation and to assess final model performance. Pipelines were implemented across 2 computational infrastructures, using AWS GPU instances and Bristol's Isambard AI Phase 2 GPU instances, the latter of which allowed larger parameter ESM-2 and ProtT5-XL models to be efficiently fine-tuned and evaluated. This project also demonstrated that Pfeature representation pipelines were tractable without specialised GPU hardware.

The main finding from this project was an effect of full fine-tuning that was pathogen-specific and directionally inconsistent with fine-tuning mode and model size. Additionally, LoRA PEFT, which aims to enable task-specific pLM transfer learning at lower computational cost, did not provide performance gains compared to using embeddings from the pre-trained pLM. Overall the results do not support a consistent benefit from pLM fine-tuning using phylogeny-aware datasets for per-residue LBCE classification prediction across the pathogens tested.

Transferring these pipelines for the LBCE classification of untested pathogens is therefore unlikely to provide reliable results. Given the mixed results between pathogens, more work is required to understand the drivers for the divergent performance. This may also help to identify the reasons for the increased embedding expressiveness with pLM fine-tuning for certain pathogens that led to improved discriminative performance by the classifier. This work could include implementation updates such as averaging embeddings of overlapping windows to improve input consistency, evaluating different batch sizes, optimising hyperparameters for pLM fine-tuning and classifier training, adding class weighting to fine-tuning, and batch shuffling during classifier training. Since evaluation metrics showed poor classifier performance with lower-level data when compared to the training evaluation in fine-tuning, other classifiers such as a random forest classifier with hyperparameter tuning, as incorporated by Leite et al. [7], could be used to identify limitations with the classifier implementation in this project, or otherwise highlight whether the pLM embeddings for these pathogens contain information that is innately challenging to classify.

This second point raises questions on the data heterogeneity for each pathogen dataset, as implied through the pathogen-specific analysis completed in this project. A more involved analysis of the datasets could be carried out to understand the provenance of the data for given proteins, e.g. whether certain proteins impact performance more than others. This could be a result of different experimental methods or antibodies used to derive positive or negative epitope labels. Furthermore, since the taxon levels for each pathogen dataset varied, comparing the extent to which different taxon levels differ between dataset levels and the impact on LBCE prediction may be helpful.

As previously shown, embedding quality can be scrutinised using visualisation approaches such as

hierarchical agglomerative clustering (HAC) and t-distributed stochastic neighbour embedding (t-SNE) [19]. These approaches would give a visual interpretation of potential clusters of sequences within the embedding space that could inform epitope classification and the effectiveness of fine-tuning. Taking this a step further, if substantial task-specific separation is observed, dimensionality reduction methods such as principal component analysis (PCA) could serve as an intermediary step between pLM embedding generation and classification to reduce the embedding dimensionality while keeping the principal sources of embedding variation.

This project tested a single training and inference run for each pathogen/model configuration controlled through random seed initialisation. Testing different random state initialisations would provide an element of variability that could be used to assess process repeatability and robustness. Additionally, completing pLM runs repeated across infrastructure types could identify effects of the infrastructure and provide evidence of robustness to compute hardware and software. Lastly, the Ankh model, tested by Schmirler et al. [13] was not included as part of this project, along with the larger size ESM-2 and ProtT5-XL models. Given the variable performance shown here, a deeper exploration of the current pipelines should be conducted before exploring additional pLMs and larger model sizes.

Overall, the results demonstrate pLM fine-tuning did not provide a consistent improvement for LBCE per-residue classification, highlighting pathogen specific factors and pipeline implementation optimisation opportunities that that should be explored before expanding the processes developed here to different pathogen types and larger or alternate pLMs.

Bibliography

[1] Mike Schutkowski and Ulrich Reineke, editors. *Epitope Mapping Protocols*, volume 524 of *Methods in Molecular Biology™*. Humana Press, Totowa, NJ, 2009.

[2] Joakim Nøddeskov Clifford, Eve Richardson, Bjoern Peters, and Morten Nielsen. AbEpiTope-1.0: Improved antibody target prediction by use of AlphaFold and inverse folding. *Science Advances*, 11(24):eadu1823, June 2025.

[3] Francisca Villanueva-Flores, Javier I. Sanchez-Villamil, and Igor Garcia-Atutxa. AI-driven epitope prediction: a systematic review, comparative analysis, and practical guide for vaccine development. *npj Vaccines*, 10(1):207, August 2025.

[4] Jodie Ashford, João Reis-Cunha, Igor Lobo, Francisco Lobo, and Felipe Campelo. Organism-specific training improves performance of linear B-cell epitope prediction. *Bioinformatics*, 37(24):4826–4834, December 2021.

[5] Feng Jiang, Yuzhi Guo, Hehuan Ma, Saiyang Na, Weizhi An, Bing Song, Yi Han, Jean Gao, Tao Wang, and Junzhou Huang. AlphaEpi: Enhancing B Cell Epitope Prediction with AlphaFold 3. In *Proceedings of the 15th ACM International Conference on Bioinformatics, Computational Biology and Health Informatics*, BCB '24, pages 1–8, New York, NY, USA, December 2024. Association for Computing Machinery.

[6] Joakim Nøddeskov Clifford, Magnus Haraldson Høie, Sebastian Deleuran, Bjoern Peters, Morten Nielsen, and Paolo Marcatili. BepiPred-3.0: Improved B-cell epitope prediction using protein language models. *Protein Science*, 31(12):e4497, 2022. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/pro.4497.

[7] Lindeberg Pessoa Leite, Teófilo Emidio de Campos, Francisco Pereira Lobo, and Felipe Campelo. Phylogeny-informed transfer learning with protein language models for epitope prediction, March 2026. ISSN: 2692-8205 Pages: 2025.04.17.649425 Section: New Results.

[8] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, March 2023.

[9] R. Prabakaran and Yana Bromberg. Quantifying uncertainty in protein representations across models and tasks. *Nature Methods*, 23(4):796–804, April 2026.

[10] Sarah M. Burbach and Bryan Briney. Improving antibody language models with native pairing. *Patterns*, 5(5), May 2024.

[11] Meng Wang, Jonathan Patsenker, Henry Li, Yuval Kluger, and Steven H. Kleinstein. Supervised fine-tuning of pre-trained antibody language models improves antigen specificity prediction. *PLOS Computational Biology*, 21(3):e1012153, March 2025.

[12] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models, June 2021.

[13] Robert Schmirler, Michael Heinzinger, and Burkhard Rost. Fine-tuning protein language models boosts predictions across diverse tasks. *Nature Communications*, 15(1):7407, August 2024.

[14] Lindeberg Pessoa Leite, Teófilo Emidio de Campos, Francisco Pereira Lobo, and Felipe Campelo. Epitopetransfer: an implementation of phylogeny-informed transfer learning for linear b-cell epitope prediction, 2024. GitHub repository.

[15] Akshara Pande, S. Patiyal, N. Sharma, Chakit Arora, and G. Raghava. Pfeature – a comprehensive tool for computing protein/peptide features and building prediction models, 2026. Computer software.

[16] Simon McIntosh-Smith, Shoukat R. Alam, and Chris Woods. Isambard-ai: A leadership class supercomputer optimised specifically for artificial intelligence. *arXiv*, 2024.

[17] Felipe Campelo, Ana Laura Grossi de Oliveira, João Reis-Cunha, Vanessa Gomes Fraga, Pedro Henrique Bastos, Jodie Ashford, Anikó Ekárt, Talita Emile Ribeiro Adelino, Marcos Vinicius Ferreira Silva, Felipe Campos de Melo Iani, Augusto César Parreiras de Jesus, Daniella Castanheira Bartholomeu, Giliane de Souza Trindade, Ricardo Toshio Fujiwara, Lilian Lacerda Bueno, and Francisco Pereira Lobo. Phylogeny-aware linear B-cell epitope predictor detects targets associated with immune response to orthopoxviruses. *Briefings in Bioinformatics*, 25(6):bbae527, September 2024.

[18] Akshara Pande, Sumeet Patiyal, Anjali Lathwal, Chakit Arora, Dilraj Kaur, Anjali Dhall, Gaurav Mishra, Harpreet Kaur, Neelam Sharma, Shipra Jain, Salman Sadullah Usmani, Piyush Agrawal, Rajesh Kumar, Vinod Kumar, and Gajendra P.S. Raghava. Pfeature: A Tool for Computing Wide Range of Protein Features and Building Prediction Models. *Journal of Computational Biology*, 30(2):204–222, February 2023.

[19] Ana F. Rodrigues, Lucas Ferraz, Laura Balbi, Pedro Giesteira Cotovio, and Catia Pesquita. Exploring the limits of pre-trained embeddings in machine-guided protein design: a case study on predicting AAV vector viability. *Scientific Reports*, 16(1):10974, March 2026.

[20] Ernest Mordret, Alexandre Hervé, Florian Tesson, Hugo Vaysset, Tyler Clabby, Arthur Loubat, Helena Shomar, Remi Planel, Rachel Lavenir, Jean Cury, and Aude Bernheim. Protein and genomic language models uncover the unexplored diversity of bacterial immunity. *Science*, 392(6793):eadv8275, April 2026.

[21] Chiho Im, Ryan Zhao, Scott D. Boyd, and Anshul Kundaje. Sequence-based TCR-Peptide Representations Using Cross-Epitope Contrastive Fine-tuning of Protein Language Models, October 2024. Pages: 2024.10.25.619698 Section: New Results.

[22] Muhammad Attique, Tamim Alkhalifah, Fahad Alturise, and Yaser Daanial Khan. Deepbce: Evaluation of deep learning models for identification of immunogenic b-cell epitopes. *Computational Biology and Chemistry*, 104:107874, 2023.

[23] Munir Akkaya, Kihyuck Kwak, and Susan K. Pierce. B cell memory: Building two walls of protection against pathogens. *Nature Reviews Immunology*, 20:229–238, 2020.

[24] Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C. Lawrence Zitnick, Jerry Ma, and Rob Fergus. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *PNAS*, 2019.

[25] Facebook Research. Esm: Evolutionary scale modeling, 2026. Software repository.

[26] Sapir Israeli and Yoram Louzoun. Single-residue linear and conformational b cell epitopes prediction using random and esm-2 based projections. *Briefings in Bioinformatics*, 25(2):bbae084, 03 2024.

[27] Ahmed Elnaggar, Michael Heinzinger, Christian Dallago, Ghalia Rehawi, Yu Wang, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Martin Steinegger, Debsindhu Bhowmik, and Burkhard Rost. Prottrans: Towards cracking the language of life's code through self-supervised learning. *bioRxiv*, 2021.

[28] Ahmed Elnaggar, Michael Heinzinger, Christian Dallago, Ghalia Rehawi, Yu Wang, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Martin Steinegger, Debsindhu Bhowmik, and Burkhard Rost. Prottrans: Toward understanding the language of life through self-supervised learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):7112–7127, 2022.

[29] Barnali Das and Dmitrij Frishman. Blmpred: Predicting linear b-cell epitopes using pre-trained protein language models and machine learning. *Computational and Structural Biotechnology Journal*, 31:94–100, 2026.

[30] Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, Jing Yi, Weilin Zhao, Xiaozhi Wang, Zhiyuan Liu, Hai-Tao Zheng, Jianfei Chen, Yang Liu, Jie Tang, Juanzi Li, and Maosong Sun. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence*, 5(3):220–235, 2023.

Appendix A

Appendix

A.1 Supplementary tables and figures

pLM/pFeature	ESM-2												ProtT5-XL			Pfeature
Parameters	8M			35M			150M			650M			1.2B			
mode	base	LoRA	full	base	LoRA	full	base	LoRA	full	base	LoRA	full	base	LoRA	full	
Bunyaviricetes	0.48	0.55	0.57	0.45	0.50	0.60	0.48	0.56	**0.67**	0.44	0.42	0.62	nan	nan	nan	0.46
Campylobacter jejuni	0.30	0.30	0.42	0.31	0.28	0.28	0.26	0.22	**0.53**	0.34	0.28	0.31	nan	nan	nan	0.53
Clostridioides difficile	0.46	0.45	0.52	0.50	0.46	0.53	0.49	0.49	0.53	0.49	0.52	0.48	nan	nan	nan	**0.58**
Dengue virus	0.56	0.57	0.53	0.57	0.56	0.54	0.58	0.58	0.53	0.58	0.58	0.53	nan	nan	nan	**0.58**
Legionellales	0.67	0.57	0.57	0.70	0.65	0.57	0.70	0.56	0.63	0.53	0.52	0.45	nan	nan	nan	**0.75**
Mycobacterium leprae	0.45	0.40	0.33	0.38	0.33	0.51	0.53	0.56	**0.60**	0.53	0.44	0.50	nan	nan	nan	0.52
Mycoplasmoidaceae	0.34	0.26	0.19	0.27	0.30	**0.82**	0.31	0.16	0.70	0.26	0.53	0.69	nan	nan	nan	0.47
Neisseria gonorrhoeae	0.26	0.21	0.16	0.30	0.30	0.25	0.32	0.25	0.17	0.21	0.45	0.24	nan	nan	nan	**0.53**
Orthoparamyxovirinae	0.51	0.54	0.57	0.59	0.50	0.40	0.33	0.44	0.49	**0.66**	0.61	0.60	nan	nan	nan	0.52
Orthopoxvirus	0.61	0.62	0.62	0.73	0.48	**0.82**	0.67	0.68	0.66	0.67	0.64	0.76	0.66	0.66	0.26	0.65
Pasteurellaceae	0.37	0.24	0.37	0.50	0.35	0.25	0.43	0.39	**0.56**	0.31	0.43	nan	nan	nan	nan	0.50
Yersinia pestis	0.67	0.62	0.56	0.70	0.67	0.24	0.71	0.64	**0.74**	0.41	0.34	0.27	nan	nan	nan	0.47

Table A.1: AUC performance across ESM-2, ProtT5-XL, and Pfeature models. The highest AUC for each pathogen is shown in bold.

Table A.2: Per-pathogen ANOVA results for mode and model size. p-values are shown before and after FDR correction.

Pathogen	F_{mode}	p_{mode}	$p_{\text{mode,FDR}}$	Reject	F_{size}	p_{size}	$p_{\text{size,FDR}}$	Reject
Bunyaviricetes	15.27	0.00	0.03	Yes	1.19	0.39	0.67	No
Campylobacter jejuni	1.06	0.40	0.48	No	0.51	0.69	0.83	No
Clostridioides difficile	2.31	0.18	0.31	No	1.59	0.29	0.61	No
Dengue virus	39.59	0.00	0.00	Yes	2.76	0.13	0.44	No
Legionellales	2.01	0.21	0.32	No	1.51	0.31	0.61	No
Mycobacterium leprae	10.45	0.01	0.03	Yes	6.31	0.03	0.17	No
Mycoplasmoidaceae	11.18	0.01	0.03	Yes	0.33	0.80	0.86	No
Neisseria gonorrhoeae	0.42	0.67	0.67	No	0.76	0.56	0.76	No
Orthoparamyxovirinae	0.56	0.60	0.65	No	2.62	0.15	0.44	No
Orthopoxvirus	1.17	0.36	0.48	No	0.78	0.57	0.76	No
Pasteurellaceae	3.65	0.11	0.21	No	0.25	0.86	0.86	No
Yersinia pestis	8.95	0.02	0.04	Yes	12.41	0.01	0.07	No

Table A.3: Pairwise comparisons between model adaptation methods for pathogens showing a significant mode effect after FDR correction.

Pathogen	Comparison	Coef.	Std. Err.	t	p	CI Low	CI Upp.	p_{HS}	Reject
Bunyaviricetes	Base vs. LoRA	-0.078	0.036	-2.186	0.072	-0.165	0.009	0.072	No
Bunyaviricetes	Full vs. LoRA	0.118	0.036	3.302	0.016	0.030	0.205	0.032	Yes
Bunyaviricetes	Full vs. Base	0.196	0.036	5.488	0.002	0.108	0.283	0.005	Yes
Dengue virus	Base vs. LoRA	0.013	0.010	1.290	0.244	-0.011	0.036	0.244	No
Dengue virus	Full vs. LoRA	-0.068	0.010	-6.980	<0.001	-0.092	-0.044	0.001	Yes
Dengue virus	Full vs. Base	-0.080	0.010	-8.270	<0.001	-0.104	-0.057	0.001	Yes
Mycobacterium leprae	Base vs. LoRA	-0.010	0.030	-0.317	0.762	-0.084	0.065	0.762	No
Mycobacterium leprae	Full vs. LoRA	0.115	0.030	3.791	0.009	0.041	0.190	0.019	Yes
Mycobacterium leprae	Full vs. Base	0.125	0.030	4.109	0.006	0.051	0.200	0.019	Yes
Mycoplasmoidaceae	Base vs. LoRA	0.050	0.101	0.496	0.638	-0.197	0.297	0.638	No
Mycoplasmoidaceae	Full vs. LoRA	0.436	0.101	4.320	0.005	0.189	0.683	0.015	Yes
Mycoplasmoidaceae	Full vs. Base	0.386	0.101	3.824	0.009	0.139	0.633	0.017	Yes
Yersinia pestis	Base vs. LoRA	0.025	0.061	0.404	0.700	-0.125	0.174	0.700	No
Yersinia pestis	Full vs. LoRA	-0.211	0.061	-3.446	0.014	-0.360	-0.061	0.027	Yes
Yersinia pestis	Full vs. Base	-0.236	0.061	-3.850	0.008	-0.385	-0.086	0.025	Yes

Table A.4: Per-pathogen ANOVA results for Pfeature, ESM-2, and ProtT5-XL.

Pathogen	F	p	p_{FDR}	Reject
Bunyaviricetes	0.700	0.421	0.631	No
Campylobacter jejuni	5.949	0.033	0.132	No
Clostridioides difficile	12.070	0.005	0.062	No
Dengue virus	1.349	0.270	0.631	No
Legionellales	0.960	0.348	0.631	No
Mycobacterium leprae	6.247	0.030	0.132	No
Mycoplasmoidaceae	0.194	0.668	0.801	No
Neisseria gonorrhoeae	2.496	0.142	0.427	No
Orthoparamyxovirinae	0.084	0.777	0.848	No
Orthopoxvirus	1.013	0.390	0.631	No
Pasteurellaceae	0.434	0.524	0.698	No
Yersinia pestis	0.001	0.972	0.972	No

Table A.5: Pairwise model comparisons by pathogen. Coefficients represent the estimated difference between the models, with corresponding standard errors, t-statistics, p-values, and Benjamini–Hochberg FDR-adjusted p-values.

Pathogen	Comparison	Coef.	Std. Err.	t	p	p_{FDR}	Reject
Bunyaviricetes	Pfeature–ESM-2	-0.084	0.100	-0.836	0.421	0.421	No
Campylobacter jejuni	Pfeature–ESM-2	0.270	0.111	2.439	0.033	0.033	Yes
Clostridioides difficile	Pfeature–ESM-2	0.142	0.041	3.474	0.005	0.005	Yes
Dengue virus	Pfeature–ESM-2	0.048	0.042	1.162	0.270	0.270	No
Legionellales	Pfeature–ESM-2	0.146	0.149	0.980	0.348	0.348	No
Mycobacterium leprae	Pfeature–ESM-2	0.229	0.091	2.499	0.030	0.030	Yes
Mycoplasmoidaceae	Pfeature–ESM-2	0.107	0.243	0.441	0.668	0.668	No
Neisseria gonorrhoeae	Pfeature–ESM-2	0.179	0.114	1.580	0.142	0.142	No
Orthoparamyxovirinae	Pfeature–ESM-2	0.050	0.173	0.290	0.777	0.777	No
Orthopoxvirus	Pfeature–ESM-2	0.008	0.135	0.056	0.956	0.956	No
Orthopoxvirus	ProtT5-XL–ESM-2	-0.118	0.084	-1.404	0.184	0.551	No
Orthopoxvirus	ProtT5-XL–Pfeature	-0.125	0.150	-0.835	0.419	0.628	No
Pasteurellaceae	Pfeature–ESM-2	0.115	0.175	0.658	0.524	0.524	No
Yersinia pestis	Pfeature–ESM-2	-0.008	0.212	-0.036	0.972	0.972	No

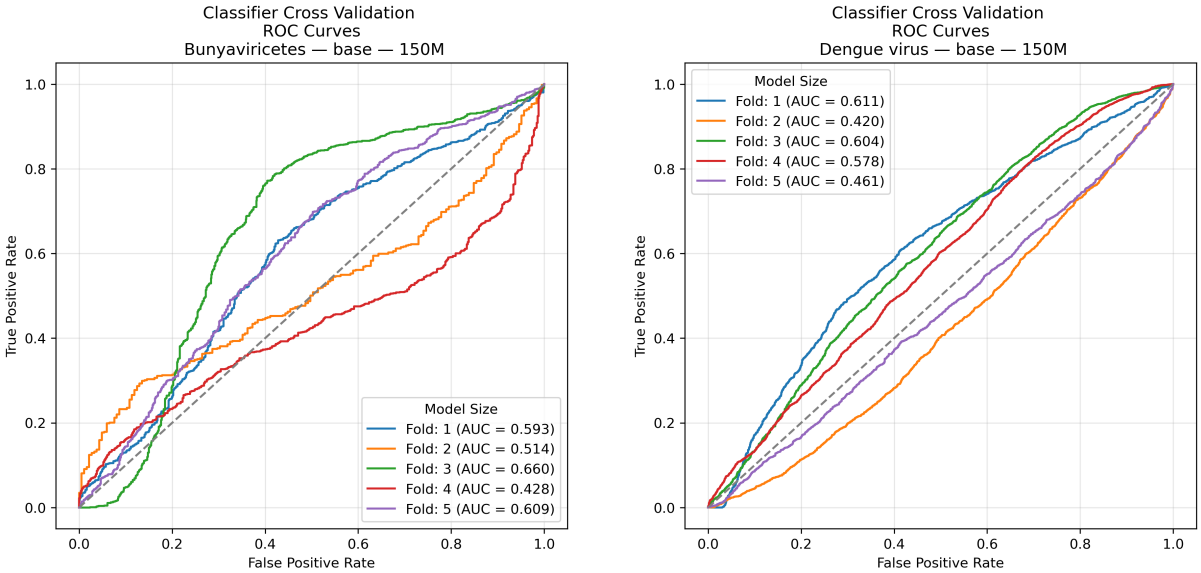


Figure A.1: Classifier cross validation evaluation ROC curves from each validation fold for the 150M ESM-2 model using pre-trained (base) pLM embeddings. Bunyaviricetes (left) and Dengue virus (right).

A.1.1 Validation of Residue and Label Alignment

The following code compares the original protein sequence and residue-level labels with the corresponding processed tokens and labels after preprocessing. This check was performed on a representative protein.

```
# Compare original and processed labels for one protein
original = train_splits['final'].iloc[1]
processed = train_datasets['final'][1]

sequence = original['sequence']
original_labels = original['label']
positions = original['position']

processed_tokens = tokenizer.convert_ids_to_tokens(processed['input_ids'])
processed_labels = processed['labels']

comparison_rows = []

for residue_idx in range(len(sequence)):

    # +1 because processed token 0 is CLS
    processed_idx = residue_idx + 1

    comparison_rows.append({
        'residue_index': residue_idx,
        'position': positions[residue_idx],
        'original_residue': sequence[residue_idx],
        'original_label': original_labels[residue_idx],
        'processed_token_index': processed_idx,
        'processed_residue': processed_tokens[processed_idx],
        'processed_label': processed_labels[processed_idx],
        'label_match': (original_labels[residue_idx] == processed_labels[processed_idx]),
        'residue_match': (sequence[residue_idx] == processed_tokens[processed_idx])})

comparison_df = pd.DataFrame(comparison_rows)

output_file = Path(f'{local_output}_label_alignment_one_protein.csv')
```

Listing A.1: Validation of residue and label alignment after dataset preprocessing for pLM fine-tuning.

The following code validates residue, position, protein identifier, and label alignment after batch processing for classifier training.

```
# Convert labels and positions to tensors
labels = [torch.tensor(x) for x in batch['label']]
positions = [torch.tensor(x) for x in batch['position']]

# Pad labels and positions to the largest sequence in the batch
labels = pad_sequence(
    labels,
    batch_first=True,
    padding_value=-100
)

positions = pad_sequence(
    positions,
    batch_first=True,
    padding_value=-100
)

# Append embeddings, labels, masks, and metadata
emb_output_list.append({
    'embeddings': embeddings,
    'labels': labels,
    'protein_id': batch['protein_id'],
    'position': positions,
    'sequence': batch['sequence']
})


# Validate alignment after batch processing
check_idx = 1

original = lower_train_all[check_idx]

original_protein_id = original['protein_id']
original_sequence = original['sequence']
original_labels = original['label']
original_positions = original['position']

batch_idx = check_idx // batch_size
item_idx = check_idx % batch_size

processed_batch = full_train_batched[batch_idx]

processed_protein_id = processed_batch['protein_id'][item_idx]
```

```
processed_positions = processed_batch['position'][item_idx]
processed_sequence = processed_batch['sequence'][item_idx]

processed_labels = (processed_batch['labels'][item_idx].tolist())
processed_labels = processed_labels[:len(processed_sequence)]

comparison_rows = []

for residue_idx in range(len(original_sequence)):
    comparison_rows.append({
        'residue_index': residue_idx,
        'original_protein_id': original_protein_id,
        'processed_protein_id': processed_protein_id,
        'original_position': original_positions[residue_idx],
        'processed_position': processed_positions[residue_idx],
        'original_residue': original_sequence[residue_idx],
        'processed_residue': processed_sequence[residue_idx],
        'original_label': original_labels[residue_idx],
        'processed_label': processed_labels[residue_idx],
        'protein_id_match': (original_protein_id == processed_protein_id),
        'position_match': (original_positions[residue_idx] == processed_positions[residue_idx]),
        'residue_match': (original_sequence[residue_idx] == processed_sequence[residue_idx]),
        'label_match': (original_labels[residue_idx] == processed_labels[residue_idx])
    })

comparison_df = pd.DataFrame(comparison_rows)

comparison_df.to_csv(Path(f'{local_output}_lower_level_label_alignment.csv'))
```

Listing A.2: Validation of residue, position, protein identifier, and label alignment after batch processing for classifier training.